TECHNICAL MASTER DOCUMENTATION

NETRA-X

Evidence-Grounded Attribution for Dark-Web Threat Actors

From first principles through mathematics, machine learning, code and production hardening — with every claim traced to the repository.

> **This document describes the implementation as it exists, not as it was designed.** Every technical claim was verified against the working tree before being written. Where documentation, docstrings or slide material disagree with the code, the disagreement is recorded explicitly rather than smoothed over. Components that are heuristic are labelled heuristic; models that were never trained are labelled untrained.

Problem statement SIH26151 · Smart India Hackathon 2026 · NTRO

Repository github.com/me13krishna/NETRA-X

Scale 12,073 LOC Python · 4,868 LOC TypeScript · 38 REST endpoints · 19 tables · 115 tests

Verified against commit 58c2f99 · 1 September 2026

VOLUME ONE

Foundations: The Problem Before The Code

> This volume assumes no prior knowledge. It explains what the dark web is, why anonymity is hard to break, what nevertheless leaks, and why those leaks make attribution possible at all. Only once that is clear does NETRA-X make sense — because NETRA-X is not a tool that breaks Tor. It is a tool that reasons carefully about mistakes people already made.

1.1 The Dark Web, Tor, and Hidden Services

What is it?

The **surface web** is what search engines index. The **deep web** is everything behind a login or a query form — your bank statement, a university portal. The **dark web** is a small subset reachable only through special software, most commonly **Tor**.

Why does it exist?

Ordinary internet traffic reveals who is talking to whom. Even with HTTPS encrypting *what* you send, your ISP still sees *that* you contacted a particular server, and the server sees your IP address. For journalists, dissidents and criminals alike, that metadata is the exposure. Tor was built to break the link between sender and destination.

How does it work?

Tor — "The Onion Router" — wraps your traffic in layers of encryption and routes it through three volunteer-run relays. Each relay peels one layer and learns only its immediate neighbours:

```
YOU ──encrypted──▶ GUARD ──encrypted──▶ MIDDLE ──encrypted──▶ EXIT ──▶ destination
     (knows you,        (knows guard         (knows middle       (knows destination,
      not the dest.)     and exit only)       and dest.)          not you)

No single relay ever sees both endpoints. That is the entire security argument.
```

A **hidden service** (or **onion service**) goes further. The server itself is also anonymous — it never reveals its IP. Client and server each build circuits to a shared *rendezvous point*, so neither learns the other's location. The address is not a domain name bought from a registrar; it is derived from the service's public key:

```
xqgo5m3u2srkq3kxclhktncfksbptbr2pui3rjhwknnv4uw6o3x2phqd.onion
└──────────────────────── 56 characters, base32 ─────────────────────────┘

v3 onion addresses encode an Ed25519 public key plus a checksum and
version byte. The address IS the key — which is why you cannot seize
it from a registrar, and why NETRA-X validates exactly 56 characters
from the alphabet [a-z2-7].
```

> **NETRA-X USAGE**
>
> The onion-address pattern is enforced in `workers/extraction/extractor.py` as `ONION_SERVICE_REGEX = re.compile(r"\b[a-z2-7]{56}\.onion\b", re.IGNORECASE)`. The strictness matters: a 54- or 55-character string is not a valid v3 address, and accepting one would put a fabricated identifier into the evidence ledger.

Why is anonymity difficult to break?

Attacking Tor directly requires either controlling a large fraction of the network (to correlate traffic timing at both ends), or exploiting the software. Both are expensive, legally fraught, and outside what a hackathon project — or most lawful investigations — can or should do.

> **THE CENTRAL DESIGN PREMISE**
>
> **NETRA-X does not attack Tor.** It never attempts traffic correlation, relay operation, or exploitation. It works exclusively on information that operators voluntarily published or accidentally exposed. This is not a limitation apologetically accepted — it is the design, and it is what makes the system lawful and its output defensible.

1.2 What Still Leaks

Anonymity systems protect the *network layer*. They do nothing about human behaviour or server misconfiguration. Operators leak in six recurring ways, and each maps to an evidence family in the engine.

Leak	What it is and why it happens	Why it identifies someone
PGP keys	Public keys published so buyers can encrypt messages. Reused across marketplaces because generating and re-establishing reputation on a new key is costly.	A fingerprint is a 160-bit hash. Two people never share one by accident. This is the strongest single signal that exists.
Crypto wallets	Payment addresses posted publicly. Bitcoin's ledger is fully public and permanent.	Address reuse links personas directly. Co-spending in one transaction proves common key control.
Infrastructure	Favicon files, TLS certificates, server banners, HTTP headers — copied wholesale when an operator clones their site to a new host.	An identical favicon on an onion service and a clearnet host is a pivot from anonymous to attributable.
Writing style	Function-word frequencies, punctuation habits, character n-grams. Largely unconscious and hard to suppress deliberately.	Weak per-sample but persistent. Useful only in aggregate and only with enough text.
Handles	Usernames. People pick memorable names and reuse or lightly mutate them.	Weak — handles collide constantly. `shadow` is not evidence of anything.
Temporal patterns	Posting times cluster into waking hours, revealing an approximate timezone.	Very weak alone. Millions share any given schedule. Useful only as corroboration.

> **WHY THIS TABLE IS THE WHOLE DESIGN**
>
> Notice the enormous spread in strength. A PGP fingerprint match is conclusive; a shared timezone is nearly worthless. Any system that treats these as comparable "links" will produce confident nonsense. The mathematics in Volume 3 exists precisely to encode this spread numerically rather than leaving it to a developer's intuition.

1.3 Core Concepts, Defined

Identifier

> Any string that points to an entity: a handle, wallet address, PGP fingerprint, onion address, email. Some are near-unique (fingerprints); some are nearly meaningless alone (a common username).

Persona

> A named online identity — one handle on one platform. An operator may run many personas. The *persona* is what you observe; the *operator* is what you want to identify.

Entity resolution

> Deciding whether two records refer to the same real-world entity. The academic field is **record linkage**, formalised by Fellegi and Sunter in 1969 for census data — matching "J. Smith, 42 Oak St" to "John Smith, 42 Oak Street" without a shared key.

Attribution

> In cybersecurity, determining who is responsible for an activity. NETRA-X performs *persona-level* attribution — linking online identities to each other — not *real-world* attribution to a named human.

OSINT

> Open-Source Intelligence: collection from publicly available sources. Distinguished from SIGINT or HUMINT by requiring no privileged access, which is what makes it lawful for a broad range of actors.

Provenance

> The complete recorded history of a piece of evidence: where it came from, when, under what legal basis, and proof it has not changed since. Without provenance, evidence is merely an assertion.

Link analysis

> Studying relationships between entities as a graph rather than as isolated records — the structural view that makes clusters and brokers visible.

1.4 The Problem NETRA-X Solves

An analyst investigating a marketplace vendor accumulates fragments: a forum post here, a PGP block there, a wallet address in a listing, a mirror site with a familiar layout. The fragments plainly relate — but "plainly" is doing dangerous work in that sentence.

The naive approach and why it fails

The obvious solution is a rule engine: if two personas share an identifier, link them.

```
if persona_a.pgp == persona_b.pgp:      link()     # correct
if persona_a.handle == persona_b.handle: link()     # frequently wrong
if same_timezone(a, b):                  link()     # almost always wrong
```

Four failure modes make this untenable:

1. **All evidence is treated as equal.** A shared fingerprint and a shared timezone both produce "link", though one is near-proof and the other is nearly noise.
2. **Evidence cannot accumulate.** Five weak signals may jointly be persuasive. A boolean rule cannot express "weak, but five of them, and independent."

3. **Correlated evidence is double-counted.** A reused wallet address and a co-spending cluster containing that address are *one* leak observed twice. Counting both doubles the apparent confidence for no new information.
4. **There is no way to express uncertainty.** The output is a link or no link. An analyst cannot triage, and a court cannot weigh it.

> THE CONSEQUENCE THAT MATTERS
>
> In attribution, a false positive is not a bug — it is an accusation against the wrong person. A system that cannot say *"the evidence is insufficient"* will instead say something false. Abstention must be a first-class output, not an error path.

The NETRA-X formulation

NETRA-X reframes linking as **probabilistic inference**. Rather than asking "do these match?", it asks: *given this specific evidence, how much more likely is it that these two personas are the same operator than that they are strangers who happen to look similar?*

That question has a formal answer — the likelihood ratio — developed in Volume 3. Every design decision downstream follows from taking it seriously.

1.5 The Twelve-Stage Pipeline

Every piece of information travels the same path. This diagram is the map for the entire document; later volumes zoom into individual stages.

```
┌──────────────────────────────────────────────────────────────────────────┐
│ 1  SOURCE          Lawful basis recorded before anything is stored       │
│                    → sources table (lawful_basis NOT NULL)               │
├──────────────────────────────────────────────────────────────────────────┤
│ 2  OBSERVATION     Raw content captured verbatim + content_hash          │
│                    → observations table                                  │
├──────────────────────────────────────────────────────────────────────────┤
│ 3  ARTIFACT        SHA-256 digest computed over exact bytes              │
│                    → artifacts table   [the anchor of all provenance]    │
├──────────────────────────────────────────────────────────────────────────┤
│ 4  EXTRACTION      Regex + parsers pull identifiers out of the bytes     │
│                    → workers/extraction/extractor.py                     │
├──────────────────────────────────────────────────────────────────────────┤
│ 5  EVIDENCE        Each identifier stored with method + dependence_group │
│                    → evidence table (append-only)                        │
├──────────────────────────────────────────────────────────────────────────┤
│ 6  CANDIDATES      Which pairs are even worth comparing?                 │
│                    → packages/attribution/candidate_gen.py   [NOT WIRED] │
├──────────────────────────────────────────────────────────────────────────┤
│ 7  FEATURES        Each shared signal → an EvidenceItem with m/u priors  │
│                    → packages/common/types.py                            │
├──────────────────────────────────────────────────────────────────────────┤
│ 8  FUSION          LLR per item → dependence discount → family caps      │
│                    → packages/attribution/fusion.py      [THE CORE]      │
├──────────────────────────────────────────────────────────────────────────┤
│ 9  CALIBRATION     LLR → posterior probability in [0,1]                  │
│                    → packages/attribution/calibration.py                 │
├──────────────────────────────────────────────────────────────────────────┤
│ 10 DECISION        HIGH / LOW / INSUFFICIENT / CONTRADICTION_REJECTED     │
│                    → packages/attribution/decide.py                      │
├──────────────────────────────────────────────────────────────────────────┤
│ 11 REVIEW          A human accepts or rejects. Machine never decides.    │
│                    → analyst_reviews + audit chain                       │
├──────────────────────────────────────────────────────────────────────────┤
│ 12 EXPORT          STIX 2.1 / PDF / CSV / JSON, provenance preserved     │
│                    → packages/evidence/stix_export.py, reporting.py      │
└──────────────────────────────────────────────────────────────────────────┘
```

Stages 1–3 establish that evidence is real and traceable. Stages 4–7 turn bytes into comparable features. Stages 8–10 are the inference engine. Stages 11–12 keep a human accountable for every conclusion that leaves the system.

Stage-by-stage input/output contract

Stage	Input	Output	Module	Principal failure mode
Source	Collection config	`Source` row	`apps/api/main.py`	Missing lawful basis → row rejected
Observation	Raw bytes	`Observation` + hash	`main.py` ingest	Duplicate content re-stored
Artifact	Bytes	SHA-256	`warc_writer.py`	Digest computed over normalised, not raw, bytes
Extraction	Text	Identifier list	`extractor.py`	Regex too loose (false IDs) or too tight (misses)
Evidence	Identifiers	`Evidence` rows	`main.py`	Wrong `dependence_group` → double counting
Candidates	Profile	Ranked pairs	`candidate_gen.py`	Recall loss: a true pair never compared
Features	Shared signals	`EvidenceItem`	`types.py`	Mis-assigned family → wrong cap

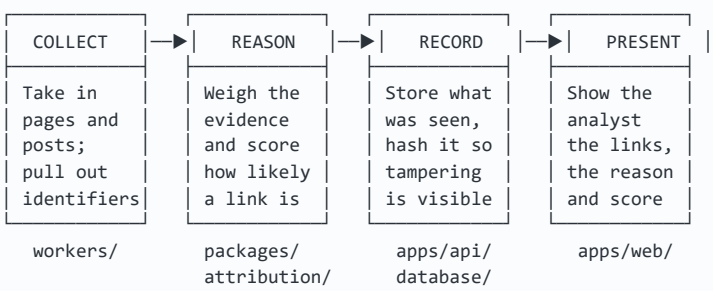
Stage	Input	Output	Module	Principal failure mode
Fusion	Items	final LLR	`fusion.py`	Correlated evidence inflating the score
Calibration	LLR	$P \in [0,1]$	`calibration.py`	Uncalibrated prior → misleading probability
Decision	P	Enum + reason	`decide.py`	Threshold mismatched to operational cost
Review	Hypothesis	Accept/reject	`main.py`	Analyst rubber-stamping the machine
Export	Hypothesis	STIX/PDF/CSV	`stix_export.py`	Provenance dropped in the export

VOLUME TWO

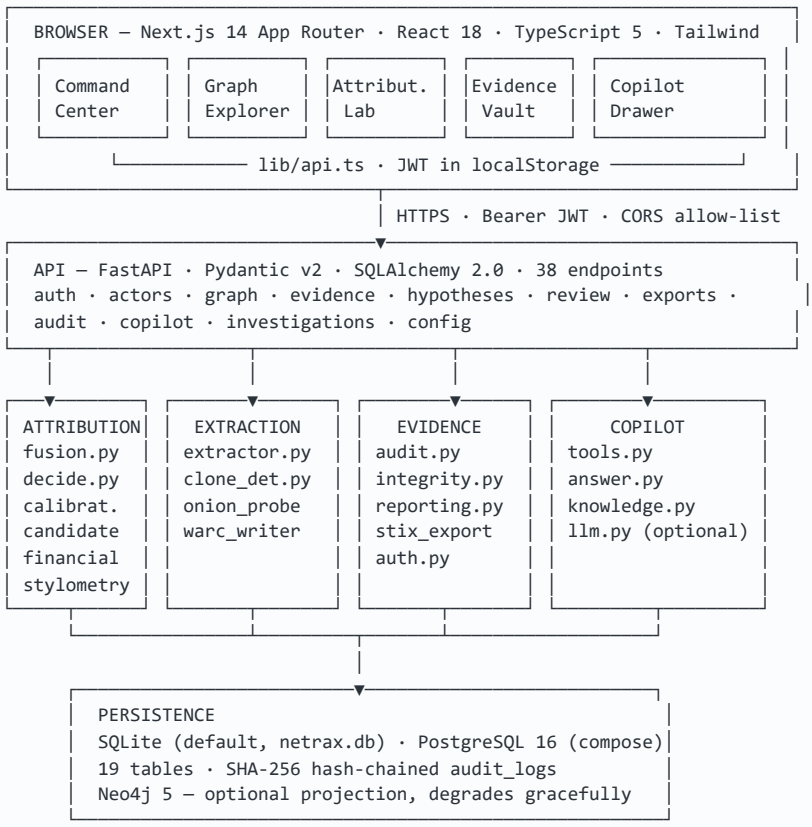
Architecture & Codebase

2.1 The Simple View

Explained to someone non-technical, NETRA-X has four parts:



2.2 The Technical View



> VERIFIED DEVIATION FROM THE PITCH MATERIAL
>
> The architecture slide shows PostgreSQL 16 as the datastore. The default runtime database is **SQLite**: `DEFAULT_DB_URL_SYNC = "sqlite:///./netrax.db"` in `apps/api/database/session.py:16`. PostgreSQL is defined in `docker-compose.yml` and is selected by setting `DATABASE_URL`, but no default path uses it. Similarly **pgvector** appears in a docstring in `candidate_gen.py`; **no vector column exists in any model**.

2.3 Repository Map

Path	LOC	Responsibility
`apps/api/`	~1,700	FastAPI application. 38 endpoints, SQLAlchemy models (19 tables), session management, startup seeding.
`apps/web/src/`	4,868	Next.js 14 frontend. 17 components, one API client, one route.
`packages/attribution/`	1,554	**The inference engine.** Fusion, calibration, decision, candidate generation, financial heuristics, graph embeddings, reporting.
`packages/evidence/`	1,108	Ledger integrity. Audit chain, auth, provenance, PDF/STIX export, referential integrity, UUIDv7.
`packages/stylometry/`	641	Authorship analysis. Feature extraction, Burrows's Delta, cosine similarity, episode gating, neural encoder.
`packages/copilot/`	~1,000	Grounded question answering over the ledger. 16 read-only tools, deterministic answerer, product knowledge, optional Claude path.
`packages/common/`	192	Shared types. `EvidenceItem`, `EvidenceFamily`, `FAMILY_CAPS`, contribution breakdown.
`packages/graph/`	259	Neo4j projection with graceful degradation when unavailable.
`packages/schemas/`	~300	Pydantic request/response contracts shared by API and tests.
`workers/extraction/`	164	Identifier extraction (regex) and SimHash clone detection.
`workers/collection/`	154	Onion probing, artifact hashing, Redis event bus.
`bench/`	~500	Synthetic benchmark generation and metric evaluation.
`seed/`	~700	Deterministic demo dataset: hero scenario plus a 14-actor network.
`tests/`	~2,000	115 tests across 18 files.

2.4 The Data Model

Nineteen tables. The critical structural idea is the **provenance chain** — you can walk from any score back to the exact bytes it came from.

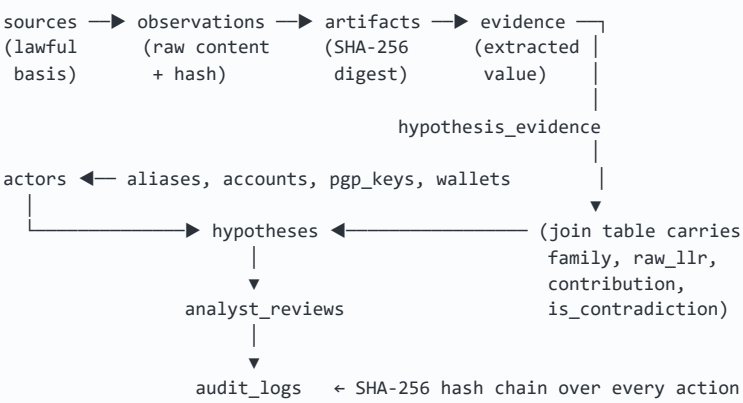


Table	Purpose and the reason it exists in this shape
`sources`	Collection origin. `lawful_basis` is **NOT NULL** — legality is a property of every observation, not an optional annotation.
`observations`	Raw captured content plus `content_hash` for duplicate detection.
`artifacts`	SHA-256 over exact bytes. **The anchor of provenance** — every evidence row points here.
`evidence`	One extracted identifier. Carries `dependence_group` (drives discounting) and retraction columns.
`actors`, `aliases`, `accounts`, `pgp_keys`, `wallets`, `onion_services`, `servers`	Entity tables. Aliases are separate from actors because one operator wears many handles — that separation *is* the product thesis.
`hypotheses`	A scored claim that two actors are one. Stores `raw_log_lr`, `calibrated_prob`, `status`, model and calibration versions.
`hypothesis_evidence`	Join table carrying `family`, `raw_llr`, `contribution`, `is_contradiction`. **This is the waterfall** — the per-item answer to "why do you believe that".
`analyst_reviews`	Human decisions. Separate from `hypotheses.status` because the decision (a verb) and the resulting state differ.
`audit_logs`	Append-only, hash-chained. Columns `seq`, `payload`, `payload_hash`, `prev_hash`, `entry_hash`.
`cases`, `case_members`, `case_identifiers`	Investigation workspace and the identifiers attached to it.

2.5 Tracing a Request End-to-End

What actually happens when an analyst opens a hypothesis:

```
1  Browser    AttributionLab.tsx mounts
                └─▶ apiFetch("/api/v1/hypotheses/{id}")
                      adds Authorization: Bearer <JWT from localStorage>

2  CORS       Origin checked against CORS_ORIGINS allow-list
                (wildcard + credentials is rejected by browsers — hence
                 an explicit list, main.py:53)

3  Auth       Depends(get_current_user)
                └─▶ decode_access_token() — PyJWT, HS256
                └─▶ 401 if signature invalid or user absent

4  Handler    get_hypothesis(id, db, user)      main.py:~880
                └─▶ joinedload(evidence_items → evidence → artifact)
                    one query, no N+1

5  Assemble   for he in h.evidence_items:
                  build EvidenceWaterfallItem
                    sha256 = ev.artifact.sha256  if present else None
                    ← previously defaulted to e3b0c442…b855, the SHA-256
                      of the EMPTY STRING, presenting absent provenance
                      as a verified digest. Now returns null.

6  Tier       calibrated_prob ≥ 0.85 → "High Confidence"
                               ≥ 0.60 → "Medium Confidence"
                               ≥ 0.35 → "Low Confidence"
                               else   → "Insufficient Evidence"

7  Response   HypothesisSchema — Pydantic validates on the way out;
              a malformed field is a 500 here, not corrupt data downstream

8  Render     EvidenceWaterfall.tsx draws one bar per family, signed
              contributions, contradictions in red, digest under each row
```

VOLUME THREE

Mathematics: Bayes, Likelihood Ratios and Fusion

> This is the intellectual core of NETRA-X. Everything here is implemented in `packages/attribution/fusion.py` (249 lines), `calibration.py` (65 lines) and `decide.py` (163 lines). We build from school-level probability up to the exact expressions the code evaluates.

3.1 Probability Foundations

Basic probability

A probability $P(A)$ is a number in [0, 1] measuring how likely event A is. $P = 0$ means impossible, 1 means certain.

Conditional probability

$P(A|B)$ — "the probability of A **given** B" — is the probability of A once you already know B happened. This is the single most important idea in the document.

$$P(A|B) = P(A \cap B) \,/\, P(B)$$

CONDITIONAL PROBABILITY

Concrete example. Pick a random person. P(uses Tor) might be 0.01. But P(uses Tor | posts on a darknet marketplace) is close to 1. The evidence changed the probability enormously. That change is exactly what NETRA-X measures.

Independence

Events A and B are **independent** if knowing one tells you nothing about the other: $P(A|B) = P(A)$. Independence matters here because it is the assumption that allows evidence to be *added* — and it is frequently violated in practice, which is why Section 3.6 exists.

3.2 Bayes' Theorem

Why does it exist?

We can usually measure P(evidence | hypothesis) — "if these really were the same person, how often would we see this?" But we want P(hypothesis | evidence) — "given what we see, are they the same person?" Bayes' theorem converts between the two.

$$P(H|E) = P(E|H) \cdot P(H) \,/\, P(E)$$

BAYES' THEOREM

Term	Name	Meaning in NETRA-X
$P(H)$	Prior	Belief that two randomly chosen personas are the same operator, *before* looking at evidence. Low — most personas are unrelated.
$P(E\|H)$	Likelihood	Probability of seeing this evidence *if* they are the same operator.
$P(E)$	Marginal	Probability of seeing this evidence at all. Usually hard to compute — which motivates the ratio form.
$P(H\|E)$	Posterior	Updated belief after weighing the evidence. **The output we want.**

A worked numerical example

> MEDICAL ANALOGY — THE BASE-RATE TRAP
>
> A disease affects 1 in 1,000. A test is 99% accurate both ways. You test positive. What is P(sick | positive)?
>
> Most people answer 99%. Consider 100,000 people: 100 are sick and 99 test positive; 99,900 are healthy and 1% — **999 people** — test positive anyway. So 1,098 positives, of whom 99 are truly sick: $P \approx$ **9%**, not 99%.
>
> **Why this governs NETRA-X:** the prior dominates when the base rate is low. Most persona pairs are unrelated, so weak evidence must not be allowed to produce a confident link. This is encoded directly as `prior_odds_log = -2.0` in `calibration.py:13`.

3.3 The Likelihood Ratio

Why a ratio?

$P(E)$ in Bayes' theorem is awkward — computing the probability of observing a given body of evidence in general is not tractable. But if we compare two competing hypotheses over the *same* evidence, that term cancels.

Define the two hypotheses:

- H_1 — the personas are the **same** operator
- H_0 — the personas are **different** operators

$$\text{LR} = P(E|H_1) \,/\, P(E|H_0)$$

LIKELIHOOD RATIO

In record-linkage terminology (Fellegi & Sunter, 1969) these are named:

Symbol	Definition	Example — PGP fingerprint match
m	P(observe this signal | same operator)	0.9999 — if it is the same operator, their fingerprint almost certainly matches

Symbol	Definition	Example — PGP fingerprint match
u	P(observe this signal | different operators)	0.00000001 — two strangers essentially never share a 160-bit fingerprint

Why logarithms?

Three reasons, all practical:

1. **Multiplication becomes addition.** Independent evidence multiplies likelihood ratios; logs turn that into a sum, which is far easier to compute, cap, discount and explain.
2. **Numerical stability.** The PGP ratio is $0.9999 / 10^{-8} \approx 10^{8}$. Chaining several such ratios overflows quickly. Logs keep values in a comfortable range.
3. **Symmetry.** Evidence for is positive, evidence against is negative, and neutral evidence is exactly zero.

$$\mathrm{LLR} = \ln(m / u)$$

LOG-LIKELIHOOD RATIO — THE ATOM OF THE WHOLE ENGINE

Reading LLR values

LLR	Likelihood ratio	Interpretation
0	1×	Evidence is uninformative — equally likely either way
+2.3	10×	Ten times more likely if same operator
+4.6	100×	Strong evidence
+18.4	10^8×	A PGP fingerprint match, uncapped
−2.3	0.1×	Evidence *against* the link

The actual implementation

packages/common/types.py:53–68 — EvidenceItem.get_effective_llr()

```
def get_effective_llr(self) -> float:
    if self.abstain:          return 0.0
    if self.is_contradiction: return 0.0
    if self.llr is not None:
        base_llr = self.llr
    else:
        u_safe = max(self.u_i, 1e-12)      # guard against div-by-zero
        m_safe = max(self.m_i, 1e-12)
        base_llr = math.log(m_safe / u_safe)

    effective_mult = max(0.0, min(1.0, self.source_reliability
                                       * self.credibility_multiplier))
    return base_llr * effective_mult
```

Line-by-line

- **Abstention returns exactly 0.0** — not a small number, not a guess. An item that abstains contributes nothing, which is the arithmetic expression of "we do not know."

- **Contradictions return 0.0 here** and are handled separately (§3.7) because they must bypass family caps.
- **The 1e-12 floor** prevents division by zero and infinite LLR. With $u = 10^{-8}$ for PGP, the raw LLR is $\ln(0.9999/10^{-8}) \approx$ **18.4**.
- **The multiplier is clamped to [0,1]**, so an unreliable source can only ever *weaken* evidence, never amplify it. A source cannot be more than fully trusted.

$$\text{LLR}_i = \ln(m_i / u_i) \times \text{reliability} \times \text{credibility}$$

EFFECTIVE ITEM LLR, AS IMPLEMENTED

3.4 The Prior Table

All m and u values live in `packages/attribution/mu_table.yaml` — **11 features and 2 contradictions**. These are expert-elicited, not learned; Volume 4 examines that honestly.

Feature	Family	m	u	LLR	Rationale
`pgp_fingerprint_exact`	EXACT_IDENTITY	0.9999	10^{-8}	**18.4**	160-bit hash; collision is negligible
`ssh_host_key_fingerprint`	EXACT_IDENTITY	0.999	10^{-7}	16.1	Host key reuse across servers
`btc_address_reuse`	FINANCIAL	0.95	10^{-5}	11.5	Direct address reuse
`btc_co_input_clustering`	FINANCIAL	0.90	10^{-4}	9.1	Co-spend implies common key control
`favicon_mmh3_hash`	INFRASTRUCTURE	0.92	10^{-4}	9.1	Identical favicon across hosts
`ssl_tls_cert_serial`	INFRASTRUCTURE	0.88	5×10^{-4}	7.5	Matching custom certificate
`simhash_clone_95`	CONTENT_NLP	0.85	10^{-3}	6.7	Near-duplicate page content
`unique_typo_signature`	CONTENT_NLP	0.75	5×10^{-3}	5.0	Idiosyncratic jargon or typo
`stylometry_burrows_delta`	STYLOMETRY	0.82	10^{-2}	4.4	Low Delta distance
`temporal_diurnal_fit`	TEMPORAL	0.70	0.10	**1.9**	Timezone overlap — deliberately weak
`handle_trigram_fuzzy`	SEMANTIC_HANDLE	0.80	0.05	2.8	Similar handle spelling

> READ THE SPREAD
>
> A PGP match (18.4) is worth roughly **ten timezone matches** (1.9 each). That single ordering is the entire argument for a probabilistic engine over a rule engine — and it is data in a YAML file, not logic buried in code, so it can be revised without touching the engine.

3.5 Evidence Accumulation

For genuinely independent evidence, likelihood ratios multiply — so log-likelihood ratios add:

$$\mathrm{LLR}_{\text{total}} = \Sigma_i \mathrm{LLR}_i$$

NAIVE ACCUMULATION — VALID ONLY UNDER INDEPENDENCE

This is where naive implementations break, and where NETRA-X does three non-obvious things: dependence discounting, family caps, and uncapped contradictions.

3.6 Dependence Discounting

The problem

Suppose an operator reuses one Bitcoin address, and that address also appears in a co-spending cluster. Two evidence items fire:

- `btc_address_reuse` → LLR 11.5
- `btc_co_input_clustering` → LLR 9.1

Naively summed: 20.6, implying evidence of roughly $10^9 : 1$. But these are not independent observations — they are **one wallet leak seen two ways**. The second item adds almost no new information.

The solution

Every evidence item declares a `dependence_group`. Within a group, the strongest item counts fully; each additional item is multiplied by $\lambda = 0.25$.

$$\text{contribution}_i = \mathrm{LLR}_{(1)} \text{ if } i = 1, \text{ else } \lambda \cdot \mathrm{LLR}_{(i)} \text{ with } \lambda = 0.25$$

DEPENDENCE DISCOUNTING, SORTED DESCENDING BY LLR

packages/attribution/fusion.py:150–162

```
for grp_id, grp_items in dep_groups.items():
    sorted_grp = sorted(grp_items,
                        key=lambda x: x.get_effective_llr(), reverse=True)
    for idx, grp_item in enumerate(sorted_grp):
        eff_llr = grp_item.get_effective_llr()
        if idx == 0:
            contrib = eff_llr            # strongest counts in full
            is_disc = False
        else:
            contrib = self.lambda_discount * eff_llr   # λ = 0.25
            is_disc = True
        group_item_contribs.append((grp_item, contrib, is_disc))
        uncapped_family_sum += contrib
```

Applied to the wallet example: 11.5 + (0.25 × 9.1) = 11.5 + 2.3 = **13.8**, not 20.6. The second observation still counts — it is corroboration — but at a quarter weight.

> HONEST NOTE ON Λ
>
> λ = 0.25 is a **hand-chosen constant**, not a fitted parameter. It encodes the judgement "a correlated second observation is worth about a quarter of an independent one." It is defensible and conservative, but it was not learned from data, and this document does not claim otherwise.

3.7 Family Caps

The problem

Even with discounting, an actor who leaks in one dimension twenty different ways could accumulate an enormous score from a *single kind* of evidence.

The solution

Each evidence family has a hard ceiling. If a family's discounted sum exceeds its cap, every contribution in that family is scaled down proportionally so the total equals the cap exactly.

$$\text{scale} = \text{cap}_f / \Sigma\,\text{contributions}_f \quad \text{(when the sum exceeds the cap)}$$

PROPORTIONAL RESCALING PRESERVES RELATIVE ORDERING WITHIN A FAMILY

Family	Cap	Why this ceiling
EXACT_IDENTITY	10.0	Cryptographic identifiers; highest trust warranted
FINANCIAL	7.5	Strong, but mixers and exchanges create genuine ambiguity
INFRASTRUCTURE	5.0	Shared hosting and templates cause real collisions
CONTENT_NLP	5.0	Text is copied between unrelated sites routinely
STYLOMETRY	3.0	Deliberately low — see the note below
TEMPORAL	2.0	Timezone overlap is very weak evidence
SEMANTIC_HANDLE	2.0	Handles collide constantly

> THE STRUCTURAL CONSEQUENCE — WORTH MEMORISING
>
> The decision threshold sits at posterior $P \geq 0.85$, which with prior −2.0 requires LLR ≈ 3.73. But the caps make it **impossible to reach high confidence from stylometry alone** (max 3.0), or from timezone alone (max 2.0). Confidence structurally *requires* corroboration across independent families. That property is enforced by arithmetic, not by developer discipline.

3.8 Contradictions

Some observations do not merely fail to support a link — they actively disprove it. Two examples are encoded:

Contradiction	Weight	Meaning
`pgp_key_conflict`	−20.0	Different PGP keys explicitly published for the same profile
`temporal_impossible_overlap`	−15.0	Simultaneous posts from geographically incompatible locations

> **WHY CONTRADICTIONS ARE UNCAPPED**
>
> Supporting evidence is capped; contradicting evidence is not. This asymmetry is deliberate and ethical. The cost of a false link (accusing the wrong person) far exceeds the cost of a missed link (an investigation continues). A −20 penalty can therefore overwhelm any amount of supporting evidence, and the system fails toward silence.

$$\text{LLR}_{\text{final}} = \Sigma_f \min(\Sigma \text{ contributions}_f, \text{cap}_f) - \Sigma\ W_c$$

THE COMPLETE FUSION EQUATION AS IMPLEMENTED

3.9 Calibration: From LLR to Probability

An LLR of 13.8 is meaningful to a statistician and meaningless to an analyst. Calibration maps it to a probability via the logistic (sigmoid) function.

$$P(H_1/E) = 1 / (1 + e^{-(\text{LLR} + prior)}) \quad prior = -2.0$$

CALIBRATION.PY:13–24

packages/attribution/calibration.py:13–24

```
def sigmoid_llr_to_prob(llr: float, prior_odds_log: float = -2.0) -> float:
    try:
        val = llr + prior_odds_log
        val_clipped = max(-50.0, min(50.0, val))   # prevent exp overflow
        return 1.0 / (1.0 + math.exp(-val_clipped))
    except OverflowError:
        return 1.0 if llr > 0 else 0.0
```

Why the prior is −2.0

A prior log-odds of −2.0 corresponds to prior odds of $e^{-2} \approx 0.135$, i.e. about a 12% prior probability that two arbitrary personas are the same operator. It shifts the curve right, so **zero evidence yields $P \approx 0.12$, not 0.5**. Without it, an evidence-free pair would look like a coin flip rather than an unlikely claim.

LLR	LLR + prior	Posterior P	Decision
0.0	−2.0	0.119	INSUFFICIENT_EVIDENCE
2.0	0.0	0.500	LOW_CONFIDENCE_LINK
3.73	1.73	**0.850**	HIGH_CONFIDENCE_LINK (boundary)
9.00	7.00	0.999	HIGH_CONFIDENCE_LINK
−5.0	−7.0	0.001	CONTRADICTION_REJECTED

3.10 Thresholds and the Decision Layer

packages/attribution/decide.py:106–129

```
if contradiction_penalty > 0.0 or final_llr < 0.0:
    decision = AttributionDecision.CONTRADICTION_REJECTED
elif posterior_prob >= 0.85:
    decision = AttributionDecision.HIGH_CONFIDENCE_LINK
elif posterior_prob >= 0.50:
    decision = AttributionDecision.LOW_CONFIDENCE_LINK
else:
    decision = AttributionDecision.INSUFFICIENT_EVIDENCE
```

Why contradictions short-circuit

The contradiction branch is evaluated **first**, before any probability comparison. If a contradiction fired at all, the outcome is rejection regardless of how much supporting evidence accumulated. The check `contradiction_penalty > 0.0` is on the penalty's *existence*, not its magnitude.

Threshold selection is an engineering decision, not a mathematical one

0.85 and 0.50 are policy choices about the relative cost of the two error types:

- **False positive** — two unrelated actors declared the same. In attribution this is an accusation against the wrong party.
- **False negative** — a genuine link missed. The investigation continues by other means.

Because the first is far more costly, the high-confidence bar is set well above 0.5, and everything between 0.50 and 0.85 is explicitly routed to human review rather than being auto-decided.

> VERIFIED CAVEAT: TWO DIFFERENT TIER SCALES EXIST
>
> `decide.py` uses **0.85 / 0.50** for the engine decision, but the API's `confidence_tier` string in `main.py` uses **0.85 / 0.60 / 0.35** for display. These are not the same scale. A hypothesis at $P = 0.55$ is `LOW_CONFIDENCE_LINK` to the engine but displays as "Low Confidence" only below 0.60 — the label and the decision can disagree in the band 0.50–0.60. This is a genuine inconsistency in the current implementation.

VOLUME FOUR

Machine Learning: An Honest Accounting

> This volume answers the questions a viva examiner asks hardest: what was trained, on what data, and what does the model actually learn? The honest answer for NETRA-X is more interesting than a fabricated one — and defending it correctly is far stronger than claiming a model that does not exist.

THE HEADLINE FINDING — STATE THIS PLAINLY IF ASKED

NETRA-X contains no trained machine-learning model in its live inference path. Verified: no `load_state_dict`, no `torch.load`, no `joblib.load`, and no `.pt`, `.pth`, `.pkl` or `.onnx` artifact anywhere in the repository.

The attribution engine is a **statistical inference system with expert-elicited parameters** — a Bayesian likelihood-ratio calculator whose m/u priors were authored by hand. Calling it "the ML model" in a viva would be a claim the code does not support.

4.1 Why Not Just Rules? (The Case for a Probabilistic Approach)

Volume 1 §1.4 showed why boolean rules fail. The point deserves restating precisely, because it explains what NETRA-X chose *instead* of ML:

Requirement	Rule engine	Supervised ML	NETRA-X (Bayesian LR)
Weigh evidence by strength	No	Yes (learned)	Yes (elicited priors)
Accumulate weak signals	No	Yes	Yes (LLR sum)
Handle correlated evidence	No	Partially	Yes (λ discount)
Express uncertainty	No	Yes	Yes (posterior)
Explain a specific decision	Yes	Usually poorly	**Yes — per-item waterfall**
Work with zero labelled data	Yes	**No**	**Yes**
Defensible in a legal setting	Partially	Difficult	Yes — auditable arithmetic

THE ACTUAL ENGINEERING ARGUMENT — MEMORISE THIS

Supervised ML requires labelled training data: thousands of persona pairs with ground truth "same operator / different operator". **That dataset does not exist and largely cannot exist** — ground truth for dark-web attribution is held by law enforcement, is classified, and arrives only after arrests. Fitting a model to synthetic labels would learn the generator's assumptions, not reality.

The Fellegi–Sunter framework was designed for exactly this situation: principled probabilistic linkage when parameters must be elicited rather than learned. Choosing it over a neural network is a **correct** engineering decision given the data reality, not a compromise.

4.2 What Is Actually Learned — Component by Component

Component	Status	What it really does
`mu_table.yaml`	`ELICITED`	11 features + 2 contradictions. Hand-authored m/u priors. **Not learned from data.**
`LLRFusionEngine`	`IMPLEMENTED`	Deterministic arithmetic. No parameters are fitted; λ = 0.25 and the caps are constants.
`IsotonicCalibrator`	`PARTIAL`	A real fittable model (scikit-learn `IsotonicRegression`) — but `.fit()` is called **only in** `bench/report.py`. The live API path never fits it, so it always falls through to the sigmoid.
Burrows's Delta	`IMPLEMENTED`	A classical statistical distance, not a learned model. Deterministic.
`NeuralStylometryEncoder`	`UNTRAINED`	A PyTorch network that is **randomly initialised and never trained**. See §4.4.
`Node2VecGraphEmbedder`	`EXPERIMENTAL`	Genuinely trains (Adam, backprop) — but from scratch, per request, 3 epochs, on a hardcoded adjacency. Not reachable from the UI.
`CandidateGenerator`	`NOT WIRED`	Implemented and tested, but **no API endpoint or UI calls it**. Only tests import it.

4.3 The Calibrator — A Real Model That Never Fits in Production

`IsotonicCalibrator` is the closest thing to genuine supervised ML in the repository. Isotonic regression fits a monotonically non-decreasing step function mapping scores to probabilities — the standard technique for probability calibration.

`packages/attribution/calibration.py:36–66`

```
def fit(self, llr_scores: List[float], labels: List[int]) -> "IsotonicCalibrator":
    if len(llr_scores) < 5:
        self.is_fitted = False          # too few points for a stable curve
        return self
    X = np.array(llr_scores, dtype=np.float64)
    y = np.array(labels,     dtype=np.float64)
    self.iso_reg = IsotonicRegression(out_of_bounds="clip", y_min=0.0, y_max=1.0)
    self.iso_reg.fit(X, y)
    self.is_fitted = True
    return self

def predict_proba(self, llr_score: float) -> float:
    if self.is_fitted and self.iso_reg is not None:
        proba = float(self.iso_reg.predict([llr_score])[0])
        return max(0.0, min(1.0, proba))
    else:
        return sigmoid_llr_to_prob(llr_score, self.prior_odds_log)   # ← always this path live
```

The training data, if it were fitted

- **Input (X)** — one raw LLR score per persona pair
- **Label (y)** — 1 if the same operator, 0 otherwise
- **Learned function** — a monotone map LLR → P

- **Source** — `bench/synthetic/scenarios.py`, generated with `seed=42`

> THE GAP BETWEEN BENCHMARK AND PRODUCT
>
> `bench/report.py:38` calls `calibrator.fit(train_scores, train_labels)`, so the reported metrics reflect a *fitted* calibrator. The API never fits one. **The benchmark therefore does not measure the code path the product runs.** The live path always uses the sigmoid with prior −2.0. This should be stated openly if asked.

4.4 The Untrained Neural Encoder

`packages/stylometry/neural.py` defines a small PyTorch network: an embedding layer, two linear layers with LayerNorm and GELU, mean-pooled to a 128-dimensional vector.

packages/stylometry/neural.py:21–33 (constructor)

```
def __init__(self, vocab_size: int = 10000, embed_dim: int = 64,
             hidden_dim: int = 128):
    super().__init__()
    torch.manual_seed(42)                      # ← weights are FIXED by a seed
    self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
    self.fc1   = nn.Linear(embed_dim, hidden_dim)
    self.norm1 = nn.LayerNorm(hidden_dim)
    self.act   = nn.GELU()
    self.fc2   = nn.Linear(hidden_dim, hidden_dim)
    self.norm2 = nn.LayerNorm(hidden_dim)
```

packages/stylometry/neural.py:64–74 (singleton factory)

```
def get_neural_encoder() -> NeuralStylometryEncoder:
    global _NEURAL_ENCODER
    if _NEURAL_ENCODER is None:
        encoder = NeuralStylometryEncoder()
        encoder.eval()          # inference mode — but on RANDOM weights
        _NEURAL_ENCODER = encoder
    return _NEURAL_ENCODER
```

> WHAT THIS ACTUALLY IS
>
> There is no `load_state_dict`. No checkpoint exists. The weights are whatever `torch.manual_seed(42)` produces. Calling `.eval()` only disables dropout and batch-norm updating — it does not make an untrained network trained.
>
> Mathematically this is a **fixed random projection** of character-n-gram indices into 128 dimensions. That is not worthless — random projections approximately preserve distances (the Johnson–Lindenstrauss lemma), so cosine similarity between two such embeddings is a crude proxy for n-gram overlap, and it is deterministic and reproducible. But it has learned *nothing* about authorship. It is closer to locality-sensitive hashing than to neural stylometry.
>
> **Correct viva answer:** "It is an untrained random projection used as a deterministic embedding placeholder. Training it would require an authorship-labelled corpus such as PAN or VeriDark, which we have not integrated."

4.5 Node2Vec — Real Training, Wrong Lifecycle

`packages/attribution/graph_embedding.py` genuinely implements training: biased random walks, a skip-gram model, Adam optimiser, backward passes.

packages/attribution/graph_embedding.py:171–180

```
optimizer = torch.optim.Adam(model.parameters(), lr=lr)
for _ in range(epochs):
    optimizer.zero_grad()
    preds = model(target_tensor, context_tensor)
    loss = criterion(preds, labels_tensor)
    loss.backward()
    optimizer.step()
```

But the lifecycle is wrong for a production system:

- Training happens **inside the request handler** (`main.py:1549` , `fit_graph_embeddings(default_adj, embed_dim=64, epochs=3)`)
- **3 epochs** — far too few for meaningful convergence
- The adjacency is a **hardcoded default**, not the live ledger graph
- Embeddings are discarded after the response; nothing is persisted
- The endpoint `/api/v1/graph/predict-link` is **not called by any frontend component**

It is best described as a **working demonstration of the technique**, not a deployed model.

4.6 Stylometry — Classical Statistics, Correctly Applied

The stylometry that *is* used is classical and sound. Features are extracted in `packages/stylometry/features.py` :

Feature	Why it signals authorship
Function-word frequencies	"the", "of", "and" — used unconsciously, topic-independent, hard to fake deliberately
Character n-grams (3–5)	Captures spelling habits and morphology below the word level
Punctuation ratios	Comma and semicolon habits are highly personal and stable
Sentence-length distribution	Rhythm of composition

Burrows's Delta

$$\Delta = (1/n)\,\Sigma_i \left| z_i^{(\mathrm{A})} - z_i^{(\mathrm{B})} \right|$$

MEAN ABSOLUTE DIFFERENCE OF Z-SCORED FEATURE FREQUENCIES

Z-scoring against a background corpus is what makes Delta work: it normalises each feature by how variable it is across authors generally, so a feature that varies wildly between everyone contributes less than one that is usually stable.

packages/stylometry/verify.py:24–40

```
def compute_burrows_delta(vec1, vec2, std_devs=None) -> float:
    if std_devs is None:
        delta = float(np.mean(np.abs(vec1 - vec2)))          # unnormalised
    else:
        z1 = vec1 / std_devs
        z2 = vec2 / std_devs
        delta = float(np.mean(np.abs(z1 - z2)))              # true Delta
    return delta
```

The abstention gate — the most defensible line in the codebase

packages/stylometry/episodes.py:12, 36–37

```
MIN_WORD_COUNT_THRESHOLD = 50   # Hard rule: abstain if text < 50 words
...
if self.word_count < MIN_WORD_COUNT_THRESHOLD:
    self.abstain = True
```

WHY THIS MATTERS MORE THAN IT LOOKS

Stylometric features are frequency estimates. On 20 words, "the" appearing twice versus once swings the estimate by 100% — the variance swamps the signal. Rather than emit a confident-looking number from noise, the system **refuses to score**. The benchmark confirms a **100% short-text abstention rate**. Combined with the STYLOMETRY family cap of 3.0, writing style can corroborate but can never carry an attribution alone.

4.7 The Benchmark — Real Numbers and Their Limits

`python -m bench.report` is reproducible. Executed against the current tree:

```
Total Test Cases Evaluated:        30
Brier Score:                       0.1007
Expected Calibration Error (ECE):  0.0044   (Target < 0.15  → PASSED)
False-Attribution Rate (FAR):      0.00%    (Target == 0.0% → PASSED)
Recall@10:                         100.00%  (Target == 100% → PASSED)
ROC-AUC Score:                     0.9028   (Target > 0.90  → PASSED)
Short-Text Abstention Rate:        100.00%
```

Metric	What it measures	Interpretation here
Brier score	Mean squared error of predicted probabilities. Lower is better; 0 is perfect.	0.1007 — moderate. See the caveat below.
ECE	Gap between stated confidence and observed accuracy. "When it says 80%, is it right 80% of the time?"	0.0044 — excellent *on this data*.
FAR	Rate of confident links between truly different actors.	0.00% — no false attributions in the set.
Recall@10	Fraction of true matches appearing in the top 10 candidates.	100% — no true pair lost to ranking.
ROC-AUC	Probability a random true pair scores above a random false pair.	0.9028 — strong separation.

THREE CAVEATS THAT MUST ACCOMPANY ANY CITATION OF THESE NUMBERS

1 — The data is synthetic and self-generated. All 30 cases come from `bench/synthetic/scenarios.py` with `seed=42`. The system is being evaluated on data produced by the same assumptions that built the scorer. This measures internal consistency, **not real-world accuracy**. No claim about field performance follows from it.

2 — Brier is reported but not gated. The exit gate at `bench/report.py:94` checks only ECE, FAR, Recall@10 and ROC-AUC. Brier (0.1007) is printed but never asserted, so the harness prints "All benchmark quality metrics passed cleanly" while a commonly cited target of < 0.08 is not met.

3 — The benchmark fits a calibrator the product does not use (§4.3).

4.8 What a Real Training Pipeline Would Require

Asked "how would you make this genuinely ML?", the answer is concrete:

```
REAL LABELS        Court records, indictments, confirmed LE attributions.
  │                Realistically hundreds — not thousands — of pairs.
  ▼
FEATURE VECTORS    One row per persona pair; columns = the 11 signals,
  │                each present/absent plus its strength.
  ▼
SPLIT              Split by ACTOR, never by pair. Splitting by pair leaks:
  │                the same actor lands in train and test, inflating scores.
  ▼
FIT m/u            Estimate m_i and u_i empirically instead of eliciting:
  │                  m_i = P(signal | same)      from positive pairs
  │                  u_i = P(signal | different) from negative pairs
  ▼
FIT λ              Learn dependence discount per group from observed
  │                correlation, rather than fixing 0.25 globally.
  ▼
CALIBRATE          Fit isotonic regression on a held-out split — and
  │                actually load it in the API path.
  ▼
EVALUATE           Brier, ECE, ROC-AUC, FAR on truly held-out actors.
  ▼
VERSION + SHIP     Persist the artifact; record model_version on every
                   hypothesis (the column already exists).
```

WHY THE CURRENT DESIGN ALREADY ANTICIPATES THIS

The `hypotheses` table already stores `model_version` and `calibration_version`. The priors are already isolated in a YAML file rather than embedded in code. Swapping elicited priors for fitted ones requires **no change to the fusion engine** — only a different `mu_table.yaml` and a loaded calibrator. That separation is the single best structural decision in the project.

VOLUME FIVE

Similarity, Graph & Attribution Mechanics

5.1 Cosine Similarity

What and why

Cosine similarity measures the angle between two vectors, ignoring their lengths. For text, that matters: a 500-word post and a 50-word post by the same author have similar feature *proportions* but very different magnitudes. Comparing direction rather than length removes document length from the comparison.

$$\cos(\theta) = (\mathbf{A} \cdot \mathbf{B}) \,/\, (\|\mathbf{A}\|\,\|\mathbf{B}\|)$$

RANGE [−1, 1]; FOR NON-NEGATIVE FREQUENCY VECTORS, [0, 1]

- **A · B** — dot product, $\Sigma a_i b_i$; large when the vectors emphasise the same features
- **‖A‖** — magnitude, $\sqrt{(\Sigma a_i^2)}$; dividing by it normalises away length
- **1.0** = identical direction · **0.0** = orthogonal (unrelated)

Worked example

```
A = [0.5, 0.3, 0.2]        B = [0.4, 0.4, 0.2]
A·B  = 0.20 + 0.12 + 0.04 = 0.36
‖A‖  = √(0.25+0.09+0.04) = √0.38 = 0.6164
‖B‖  = √(0.16+0.16+0.04) = √0.36 = 0.6000
cos  = 0.36 / (0.6164 × 0.6000) = 0.9734   → very similar style
```

NETRA-X usage: `compute_cosine_similarity()` in `packages/stylometry/verify.py:13`, applied to function-word frequency vectors and to neural embeddings.

5.2 The Stylometry Decision Bands

`packages/stylometry/verify.py:79–96`

```
cos_sim    = compute_cosine_similarity(fvec1, fvec2)
delta_dist = compute_burrows_delta(fvec1, fvec2, std_devs=background_std_devs)

if   cos_sim > 0.85 or delta_dist < 0.75:   # strong same-author signal
    ...
elif cos_sim > 0.65 or delta_dist < 0.95:   # moderate signal
    ...
```

Note the `or`: either high cosine similarity *or* low Delta distance suffices. The two measures capture overlapping but distinct aspects, and requiring both would reject genuine matches where one metric is noisy. The resulting evidence item is still capped at 3.0 by the STYLOMETRY family cap.

5.3 Jaccard Similarity and Graph Topology

$$J(A,B) = |N(A) \cap N(B)| \;/\; |N(A) \cup N(B)|$$

NEIGHBOUR OVERLAP — RANGE [0, 1]

$N(A)$ is the set of nodes adjacent to A. The intuition is social: two personas who interact with the same forums, vendors and services are more likely to be one operator than two personas with disjoint circles.

Worked example

```
N(A) = {forum1, vendor2, market3}
N(B) = {forum1, vendor2, market9}
∩ = {forum1, vendor2}                    → 2
∪ = {forum1, vendor2, market3, market9}  → 4
J = 2/4 = 0.50                           → above the 0.20 threshold
```

NETRA-X usage: `CandidateGenerator.graph_topological_similarity()`, `candidate_gen.py:149-166`, with `topological_match = jaccard >= 0.20`.

VERIFIED DOCSTRING/IMPLEMENTATION MISMATCH

`multi_modal_candidate_search()` documents itself as "combining exact PGP, fuzzy handle, vector similarity, favicon hash, **and graph topology**." The implementation contains only **four** steps — PGP, handle, vector, favicon. **Graph topology is never invoked** in the unified search. The function exists and is tested independently, but the pipeline described in the docstring is not the pipeline that runs.

5.4 Candidate Fusion — Max, Not Weighted Sum

CORRECTING A COMMON ASSUMPTION

Design material describes candidate scoring as a weighted sum, $S = w_1 S_{\text{vector}} + w_2 S_{\text{graph}} + \ldots$ **No such formula exists in the codebase.** There are no weights.

packages/attribution/candidate_gen.py:214-217 (pattern repeats per signal)

```
results_by_actor[aid]["matches"].append(m)
results_by_actor[aid]["max_score"] = max(results_by_actor[aid]["max_score"],
                                         m["score"])
...
ranked.sort(key=lambda x: x["max_score"], reverse=True)
```

$$S_{\text{candidate}} = \max(S_{\text{pgp}}, S_{\text{handle}}, S_{\text{vector}}, S_{\text{favicon}})$$

THE ACTUAL CANDIDATE RANKING FUNCTION

Why max is defensible here: candidate generation is a *recall* stage. Its job is to ensure no true pair is missed before the expensive fusion step; precision is fusion's job. A single strong signal is sufficient reason to

consider a pair, and max guarantees a strong signal is never diluted by absent ones — which a weighted average would do.

5.5 SimHash — Near-Duplicate Detection

Cryptographic hashes are designed so one changed bit produces a completely different digest. For near-duplicate detection you want the opposite: similar documents should produce similar hashes. SimHash provides that.

```
1. Tokenise the document.
2. Hash each token to 64 bits (MD5, truncated — see clone_detector.py:18).
3. For each of the 64 bit positions, keep a running sum:
      bit is 1 → +1        bit is 0 → −1
4. Final fingerprint bit = 1 if the sum is positive, else 0.
5. Similarity = 1 − (Hamming distance / 64).

Changing a few tokens moves only a few of the 64 sums across zero,
so the fingerprint changes only slightly.
```

NETRA-X usage: `SimHashCloneDetector` in `workers/extraction/clone_detector.py`. Feeds the `simhash_clone_95` feature (LLR 6.7) when similarity ≥ 95%.

> **TERMINOLOGY CORRECTION FOR THE PITCH DECK**
>
> Presentation material labels this "MinHash (SimHash)". These are **different algorithms**: MinHash estimates Jaccard similarity over sets; SimHash estimates cosine similarity over weighted features via Hamming distance. **MinHash appears nowhere in the repository.** The implementation is SimHash, and the slide should say so.

5.6 The Favicon Pivot — The Highest-Value Technique

This is the single most operationally interesting method in the system, because it is what actually moves an investigation from anonymous to attributable.

```
An operator runs a hidden service. The onion address reveals nothing.
But the site serves a favicon.ico — and when they deploy a clearnet
mirror or a staging box, they copy the whole web root, favicon included.

   onion service  ─── favicon.ico ──▶  MurmurHash3 = 297738526
                                                 │
   clearnet host  ─── favicon.ico ──▶  MurmurHash3 = 297738526
                                                 │
                                                 ▼
                        Shodan query:  http.favicon.hash:297738526
                                                 │
                                                 ▼
                      Real IP address, ASN, hosting provider

The anonymity of the .onion is intact. The operator's OPERATIONAL
SECURITY is what failed — they reused a file.
```

MurmurHash3 is used because it is the hash Shodan indexes, making results directly queryable against a public dataset. It is non-cryptographic and fast — appropriate here, since the goal is a lookup key, not tamper

resistance.

NETRA-X usage: `OnionProbeEngine.inspect_favicon()` in `workers/collection/onion_probe.py:16` ; the `favicon_mmh3` column on `onion_services` ; and the `/api/v1/graph` and infrastructure views, which surface a favicon hash shared by more than one service. Feature `favicon_mmh3_hash` , LLR 9.1.

5.7 Financial Analysis — UTXO Co-Spend Clustering

The foundational heuristic of blockchain forensics: **if a single Bitcoin transaction spends inputs from several addresses, one entity controlled the private keys for all of them**, because a valid transaction requires signing every input.

```
Transaction T1:  inputs [addr_A, addr_B] → output [addr_X]
Transaction T2:  inputs [addr_B, addr_C] → output [addr_Y]

T1 ⇒ one entity controls A and B
T2 ⇒ one entity controls B and C
Transitively ⇒ {A, B, C} is one cluster.

Implemented as union-find over the input sets in
packages/attribution/financial.py.
```

Two features draw on this: `btc_address_reuse` (LLR 11.5) and `btc_co_input_clustering` (LLR 9.1). Critically, both share the dependence group `wallet_cluster_btc` , so the λ discount applies — they are one wallet leak, not two.

Monero and the hard abstention

> THE MOST PRINCIPLED DECISION IN THE CODEBASE
>
> Monero uses ring signatures, stealth addresses and confidential transactions. The co-spend heuristic **does not apply** — there is no sound way to cluster XMR addresses by this method. NETRA-X detects XMR addresses and then **refuses to score them** (`xmr_abstain` , `workers/extraction/extractor.py:35`).
>
> Applying the Bitcoin heuristic to Monero would produce confident, entirely unfounded links. Choosing to abstain rather than produce a number is exactly the behaviour the whole system is built around — and it is a strong answer to "where does your system refuse to work?"

5.8 Identifier Extraction

`workers/extraction/extractor.py:16–22`

```
PGP_FINGERPRINT_REGEX = re.compile(r"\b(?:[A-Fa-f0-9]{4}\s?){10}\b|\b[A-Fa-f0-9]{40}\b")
BTC_ADDRESS_REGEX     = re.compile(r"\b(bc1[a-z0-9]{38,59}|[13][a-km-zA-HJ-NP-Z1-9]{25,34})\b")
ETH_ADDRESS_REGEX     = re.compile(r"\b0x[a-fA-F0-9]{40}\b")
XMR_ADDRESS_REGEX     = re.compile(r"\b(4[0-9AB][1-9A-HJ-NP-Za-km-z]{93}|8[0-9AB][1-9A-HJ-NP-Za-km-z]
{93})\b")
ONION_SERVICE_REGEX   = re.compile(r"\b[a-z2-7]{56}\.onion\b", re.IGNORECASE)
EMAIL_REGEX           = re.compile(r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b")
HANDLE_REGEX          = re.compile(r"(?:^|\s)@([A-Za-z0-9_]{3,30})\b")
```

Pattern	Design detail worth defending
PGP	Two alternations: ten space-separated 4-hex groups (display format) or a bare 40-hex string. Both are the same 160-bit value written differently.
BTC	Handles bech32 (`bc1…`) and legacy Base58 (`1…` / `3…`). The Base58 class excludes `0OIl` — characters omitted from the alphabet precisely to avoid visual ambiguity.
XMR	Enforces the exact 95-character length. Detected only to trigger abstention.
Onion	**Exactly 56** characters from `[a-z2-7]` — the v3 base32 alphabet. Anything else is not a valid address.

> **KNOWN LIMITATION — REGEX EXTRACTION IS BRITTLE**
>
> Regex cannot understand context. `ETH_ADDRESS_REGEX` matches any 40-hex string prefixed `0x` — including a truncated SHA-1 or a memory address in a pasted stack trace. Nothing validates the EIP-55 checksum, and no Base58Check verification is performed on BTC addresses. Both are straightforward improvements that are simply not implemented.

VOLUME SIX

Technology Decisions and "Why Not X?"

6.1 The Complete Stack

Technology	What it is	Why NETRA-X uses it	Alternatives	Trade-off accepted	Status
Python 3.13	Interpreted language	Scientific stack (NumPy, scikit-learn, PyTorch) is unmatched; the whole engine is numerical	Go, Java, Rust	Slower execution; GIL limits CPU threading	IMPL
FastAPI	Async web framework	Pydantic validation is native, so the API contract is enforced at the boundary; OpenAPI docs are automatic	Flask, Django, Express	Younger ecosystem than Django	IMPL
Pydantic v2	Validation library	Rejects malformed data at the edge rather than deep in the engine	marshmallow, attrs	v1→v2 migration churn	IMPL
SQLAlchemy 2.0	ORM	Same models run on SQLite and PostgreSQL — dev/prod parity without rewriting queries	Raw SQL, Tortoise, Peewee	Abstraction can hide N+1 queries	IMPL
SQLite	Embedded database	**The actual default.** Zero setup — clone and run, which matters for a demo	PostgreSQL	Weak concurrent writes; no native JSON/vector types	DEFAULT
PostgreSQL 16	RDBMS	Defined in `docker-compose.yml` for deployment	MySQL, CockroachDB	Requires a running server	OPT-IN
Neo4j 5	Graph database	Multi-hop traversals are natural in Cypher, painful in SQL	NetworkX, ArangoDB	Another service to operate; degrades gracefully when absent	OPTIONAL
Next.js 14	React framework	App Router, file routing, production build pipeline out of the box	Vite+React, SvelteKit	Heavier than a plain SPA	IMPL
TypeScript 5	Typed JavaScript	API response shapes are checked at compile time	JavaScript	Build step; type maintenance	IMPL
Cytoscape.js	Graph rendering	Purpose-built for network visualisation; handles hundreds of nodes with real layout algorithms	D3, vis.js, Sigma	Large bundle; imperative API	IMPL
Tailwind CSS	Utility CSS	Consistent design tokens; no cascade conflicts across 17 components	CSS Modules, styled-components	Verbose class strings	IMPL
PyJWT	JWT library	Stateless auth; no server-side session store	Sessions, OAuth2	Tokens cannot be revoked before expiry	IMPL
scikit-learn	ML library	Only for `IsotonicRegression` in the calibrator	SciPy, custom	Large dependency for one class	PARTIAL
PyTorch	Deep learning	Neural stylometry and Node2Vec — both experimental	TensorFlow, JAX	Very heavy; **optional extra**	EXP

Technology	What it is	Why NETRA-X uses it	Alternatives	Trade-off accepted	Status
NumPy	Numerics	Vectorised feature maths in stylometry	Pure Python	None material	IMPL
ReportLab	PDF generation	Signed attribution reports	WeasyPrint, wkhtmltopdf	Imperative layout API	IMPL
PyYAML	Config parsing	Reads `mu_table.yaml` so priors are data, not code	JSON, TOML	`safe_load` required	IMPL
Redis	In-memory store	Event bus in `workers/collection/event_bus.py`	RabbitMQ, Kafka	Only used in that one module	MINIMAL
MinIO	Object storage	In `docker-compose.yml` for artifact blobs	S3, filesystem	**No project code imports it**	PLANNED
Docker	Containerisation	Reproducible multi-service environment	Bare metal, Vagrant	Local resource overhead	IMPL
pgvector	Vector index	Mentioned in a docstring only	FAISS, Qdrant	**No vector column exists**	NOT IMPL
GitHub Actions	CI/CD	Shown on the tech-stack slide	GitLab CI, Jenkins	**No `.github/workflows/` exists**	NOT IMPL

6.2 "Why Not X?" — Defending Every Choice

Q Why Python and not Go or Java?

A The system is fundamentally numerical: log-likelihood computation, vector similarity, z-scoring, isotonic regression. Python's scientific ecosystem — NumPy, scikit-learn, PyTorch — has no equivalent in Go, and matching it in Java would mean substantially more code for the same maths. The trade-off is execution speed, which is not the bottleneck: the expensive operations are database queries and network I/O, and the fusion arithmetic for one hypothesis is microseconds. If candidate generation over millions of pairs became the bottleneck, that specific stage — not the whole system — would be the rewrite candidate.

Q Why not use an LLM to decide attribution?

A Three disqualifying reasons. **Explainability** — an LLM cannot show which evidence produced which portion of a score; the evidence waterfall is the product's core deliverable and an LLM cannot produce one. **Reproducibility** — the same input must always yield the same score for an audit trail to mean anything. **Hallucination** — a system whose central promise is "never a black-box guess" cannot rest on a component that invents plausible text. The copilot in `packages/copilot/` deliberately inverts this: the LLM path, when enabled, is given *tools* and no facts, so every claim must come back from a real ledger query, and the deterministic path is the default.

Q Why not a neural network for the whole scoring problem?

A No labelled training data exists (§4.1). Beyond that, a neural scorer would forfeit the property that makes this system defensible: each evidence item's contribution is individually inspectable and independently justifiable. In a legal or intelligence context, "the network output 0.91" is not a finding — "a PGP fingerprint match contributed 10.0, capped, and a wallet co-spend contributed 2.3 after dependence discounting" is.

Q Why SQLite as the default when the slide says PostgreSQL?

A Honest answer: developer and demo experience. SQLite requires no server, so `git clone` followed by `uvicorn` produces a working system. SQLAlchemy keeps the models identical, so switching is a `DATABASE_URL` change. The trade-off is real — SQLite serialises writes and lacks native JSON and vector types — so production would use PostgreSQL. **The slide currently overstates the default and should be corrected.**

Q Why Bayesian likelihood ratios rather than a weighted score?

A A weighted score requires inventing weights with no principled meaning, and its output is uninterpretable — "72 points" answers nothing. The likelihood ratio has a precise definition: how many times more likely this evidence is under "same operator" than under "different operators". It composes correctly under independence, it converts to a probability through a single well-understood function, and it has an established legal precedent in forensic science (the "likelihood ratio framework" used in DNA evidence).

Q Why cap evidence families at all? Isn't discarding information wrong?

A The caps exist to prevent one *type* of evidence from dominating. Twenty correlated infrastructure signals should not outweigh independent corroboration across families, because they largely reflect a single underlying fact. The caps encode the principle that **confidence requires diversity of evidence**, and they enforce it arithmetically rather than relying on an analyst noticing. Information is not discarded — every item still appears in the waterfall with its scaled contribution shown.

Q Why is candidate generation needed if it isn't even wired up?

A Comparing every pair is $O(n^2)$. At 14 actors that is 91 pairs — trivial, which is exactly why it is not wired in yet. At 100,000 actors it is 5×10^9 pairs, which is infeasible. Candidate generation exists so that only plausible pairs reach the expensive fusion stage. It is implemented and tested but not yet on the request path, because the current data volume does not require it. That is a defensible sequencing decision, provided it is stated rather than implied otherwise.

Q Why Cytoscape.js rather than D3?

A D3 is a general visualisation toolkit — you build a graph renderer with it. Cytoscape.js *is* a graph renderer, with force-directed and concentric layouts, node/edge styling, and interaction handling already solved. Since the requirement is specifically network visualisation, Cytoscape removes work D3 would have required. The cost is bundle size and a more imperative API.

6.3 Three Alternative Architectures

Architecture A — Pure rule engine

Design: boolean matching rules; any shared identifier creates a link.
Strengths: trivially explainable, fast, no parameters to justify.
Fatal weakness: cannot rank, cannot accumulate weak evidence, cannot express uncertainty, cannot handle correlation. Would link every persona sharing a common handle.

Architecture B — End-to-end supervised ML

Design: featurise pairs, train a gradient-boosted classifier on labelled same/different pairs.
Strengths: learns real signal weights and feature interactions from data; likely the most accurate *if* data existed.
Fatal weakness: the labelled dataset does not exist. Additionally, per-item attribution of the score is approximate (SHAP) rather than exact, which weakens the evidentiary story.

Architecture C — Bayesian evidence fusion with human review SELECTED

Design: elicited m/u priors, LLR fusion with dependence discounting and family caps, calibration to a posterior, thresholds, mandatory analyst decision.
Strengths: works with zero labelled data; every contribution is exactly attributable; abstention is natural; parameters live in a YAML file and can be replaced with fitted values without touching the engine.
Weaknesses accepted: priors are expert judgement and could be miscalibrated; λ and the caps are hand-set; accuracy on real data is unmeasured.

> WHY C WAS CORRECT
>
> Architecture C is the only one that satisfies the actual constraints simultaneously: no training data available, explainability legally required, and abstention operationally mandatory. It also **degrades into B** — the moment real labels exist, the same engine runs with fitted priors. Choosing C does not foreclose B; choosing B would have required data that does not exist.

VOLUME SEVEN

Security, Testing, Limitations & Production

7.1 Authentication

packages/evidence/auth.py:38–42

```
def hash_password(password: str) -> str:
    salt = os.urandom(16)
    pwd_hash = hashlib.pbkdf2_hmac('sha256', password.encode('utf-8'), salt, 100000)
    return f"pbkdf2:sha256:100000${salt.hex()}${pwd_hash.hex()}"
```

PBKDF2-HMAC-SHA256 with a random 16-byte salt and 100,000 iterations. The iteration count is the point: it makes each guess expensive, so an attacker with the hash database still cannot brute-force quickly. The salt ensures identical passwords produce different hashes, defeating rainbow tables.

Tokens are JWTs signed HS256 via PyJWT. Every protected endpoint depends on `get_current_user`, which decodes the token and confirms the user still exists.

VERIFIED SECURITY GAPS — DO NOT CLAIM THESE ARE HANDLED

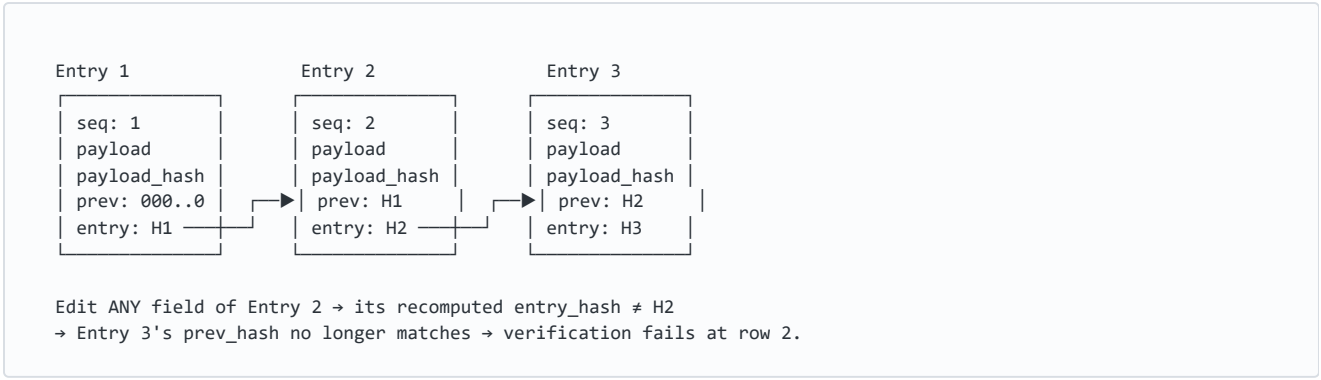
- **Ephemeral secret key.** If `SECRET_KEY` is unset, `auth.py:31` generates a random key at startup and prints a warning. Every restart invalidates all tokens; multiple replicas cannot validate each other's tokens.
- **No token revocation.** Stateless JWTs remain valid until expiry. A compromised token cannot be cancelled.
- **No rate limiting.** The login endpoint accepts unlimited attempts. Brute-force protection does not exist.
- **No RBAC enforcement.** A `role` column exists on `users` and is issued in the token, but **no endpoint checks it**. Any authenticated user can do anything.
- **No MFA.** `mfa_enabled` is stored and returned but never verified during login.
- **Seeded credentials.** Demo accounts with known passwords are created at startup — acceptable for a demo, unacceptable in production.

7.2 The Audit Chain

This is the strongest security property in the system and it genuinely works.

$$\text{entry\_hash} = \text{SHA256}(\text{seq} \,\|\, \text{prev\_hash} \,\|\, \text{actor} \,\|\, \text{action} \,\|\, \text{resource} \,\|\, \text{payload\_hash} \,\|\, \text{created\_at}_{\mu s})$$

PACKAGES/EVIDENCE/AUDIT.PY:97–115



Why the design changed — and why it matters

An earlier version only checked that `prev_hash` values linked up. It never recomputed a hash from stored data, and the payload was discarded after hashing. That meant editing `action`, `actor_user_id` or `created_at` was **undetectable**, and there was nothing to audit because the payload was gone. Three columns fixed it:

- `seq` — monotonic and **UNIQUE**. Ordering previously depended on `created_at`, so clock skew could reorder the chain and break verification on correct data. Uniqueness also makes concurrent writers collide loudly instead of silently forking the chain.
- `payload` — retained. You cannot audit what you did not keep, and `payload_hash` cannot be re-derived without it.
- `entry_hash` — computed over the row's own fields *plus* `prev_hash`, so any edited field breaks the chain from that point forward.

> PRECISE TERMINOLOGY — A LIKELY VIVA TRAP
>
> This is **tamper-evident**, not **tamper-proof**. Anyone with write access to the database can delete rows or rebuild the entire chain from scratch. What they cannot do is alter one historical row and leave the chain verifying. Real tamper-*proofing* requires external anchoring — publishing periodic root hashes to an append-only third-party log. Presentation material saying "tamper-proof" or "admissible in court" overstates what the code provides.

7.3 Ledger Integrity and Retraction

The ledger claims to be append-only. That claim was previously false: a hard `DELETE` endpoint physically removed `Evidence` rows while the `hypothesis_evidence` rows citing them remained. The measured damage was **36 of 69 citations dangling across 13 of 14 hypotheses** — scores computed from evidence that no longer existed.

Deletion is now retraction:

Column	Purpose
`retracted_at`	When the item was withdrawn. NULL means active.
`retracted_by`	Which analyst withdrew it — accountability.
`retraction_reason`	Why. Recorded in the audit chain alongside the action.

`packages/evidence/integrity.py` provides `check()`, `repair()`, `active_evidence_items()` and `rescore_hypothesis()`. Retracted evidence stays visible in the vault, struck through, and stops contributing to

scores — the difference between withdrawing a claim and pretending it was never made.

7.4 Testing

115 tests across 18 files. Full suite runs in roughly 12 seconds.

File	Tests	Covers
`test_copilot.py`	20	Grounding, abstention, entity resolution, product knowledge
`test_backend.py`	17	API endpoints, auth, schema contracts
`test_audit_chain_integrity.py`	13	Hash chain, tamper detection, sequence uniqueness
`test_fusion.py`	7	LLR computation, λ discounting, family caps
`test_ingestion.py`	7	Extraction, lawful basis, XMR abstention
`test_ledger_integrity.py`	6	Retraction, dangling citations, rescoring
`test_case_identifiers.py`	5	Persistence, dedup, audit trail
`test_reporting.py`, `test_extraction_workers.py`	10	PDF/waterfall formatting, regex extractors
`test_candidate_gen.py`, `test_graph_embedding.py`, `test_neural_stylometry.py`, `test_stylometry.py`, `test_synthetic.py`	20	Similarity functions, embeddings, scenarios
`test_review_status.py`	3	ACCEPT → ACCEPTED mapping; no bare verbs as status
`test_e2e_hero.py`	1	Full pipeline: login → ingest → score → review → export

Notable missing coverage

- No frontend tests at all — no Jest, no React Testing Library, no Playwright specs
- No load or concurrency testing
- No security tests (authz bypass, injection, token forgery)
- No property-based testing of the fusion maths (Hypothesis would suit it well)
- Calibration is evaluated only on synthetic data

7.5 Limitations — Stated Without Softening

Data and evaluation

- **Every actor in the database is synthetic.** The `actors` table has an `is_synthetic` flag and it is true for all seeded rows. The system has never processed a real dark-web corpus.
- **All metrics are on self-generated data.** ROC-AUC 0.9028 measures internal consistency, not field accuracy.

- **The m/u priors are unvalidated.** They are plausible expert judgements. Whether $u = 10^{-4}$ is the true coincidence rate for favicon collisions is unknown.

Method

- **λ = 0.25 is uniform.** Real dependence structure differs per group; one constant cannot be right everywhere.
- **Dependence groups are declared, not detected.** If an ingestion path assigns the wrong group, correlated evidence is counted as independent and the score inflates silently.
- **Independence across families is assumed.** Infrastructure and content signals are plausibly correlated (a cloned site shares both), and nothing discounts across families.
- **Regex extraction has no semantic validation** — no EIP-55 or Base58Check verification.
- **Two inconsistent tier scales exist** (Volume 3 §3.10).

Adversarial

- **The entire model assumes non-adaptive adversaries.** An operator who knows the feature list can defeat it: fresh PGP key per persona, no address reuse, deliberate style variation, distinct favicons.
- **Deliberate false-flagging is possible.** An adversary can copy another actor's favicon or post their wallet address to manufacture a link. Nothing detects planted evidence.
- **No temporal decay.** Infrastructure evidence from three years ago counts the same as evidence from yesterday, though hosting changes constantly.

Operational

- No RBAC enforcement, rate limiting, or token revocation (§7.1)
- SQLite serialises writes — unsuitable for concurrent analysts
- No CI: `.github/workflows/` does not exist
- MinIO and pgvector appear in configuration and docstrings but not in code
- Five endpoints return synthetic data from literals and are unreachable from the UI: `financial/utxo-clusters`, `attribution/waterfall`, `attribution/evaluate`, `stylometry/neural`, `graph/predict-link`

Legal and ethical

- **Attribution is probabilistic and can be wrong.** A 0.92 posterior is not proof.
- **The output is persona-linkage, not identification of a human being.**
- **"Admissible in court" is not a claim this system supports.** Admissibility depends on jurisdiction, collection legality, chain of custody and expert testimony — not on a hash chain existing.

7.6 MVP → Production Gap

Area	Implemented today	Required for production
Database	SQLite default	PostgreSQL with connection pooling, read replicas, migrations (Alembic)

Area	Implemented today	Required for production
Ingestion	Synchronous HTTP endpoint	Queue-backed workers (Celery/RQ), retries, backpressure, dead-letter handling
Auth	JWT + PBKDF2	RBAC enforcement, MFA verification, token revocation, rate limiting, rotated secrets
Models	Elicited priors, unfitted calibrator	Fitted priors from real labels, persisted calibrator, versioned artifacts, drift monitoring
Scale	All-pairs comparison	Wire candidate generation; blocking/indexing to avoid $O(n^2)$
Storage	DB rows only	MinIO/S3 for raw artifacts with lifecycle and retention policy
Observability	Audit chain, `print()`	Structured logging, metrics (Prometheus), tracing (OpenTelemetry), alerting
CI/CD	None	GitHub Actions: tests, lint, type-check, security scan, build, deploy
Testing	115 backend tests	Frontend tests, load tests, security tests, property-based fusion tests
Graph	Optional Neo4j	Required, with a maintained projection and consistency checks
Recovery	None	Backups, PITR, tested restore, disaster-recovery runbook

VOLUME EIGHT

Reference: Truth Table, Formulas, Glossary, Viva Bank

8.1 Implementation Truth Table

The mandatory honest inventory. `IMPLEMENTED` works end to end · `PARTIAL` works with caveats · `EXPERIMENTAL` runs but is not production-shaped · `MOCK` looks real but is not · `NOT IMPL` claimed somewhere but absent.

Feature	Status	File / module	Notes
LLR evidence fusion	`IMPL`	`attribution/fusion.py`	Complete: discounting, caps, contradictions, per-item contributions
Dependence discounting	`IMPL`	`fusion.py:150-162`	λ = 0.25, hand-set constant
Family caps	`IMPL`	`common/types.py:23-31`	7 families, proportional rescaling
Contradiction penalties	`IMPL`	`fusion.py` , `mu_table.yaml`	Uncapped by design; 2 defined
Sigmoid calibration	`IMPL`	`calibration.py:13`	The live path, prior −2.0
Isotonic calibration	`PARTIAL`	`calibration.py:36`	Implemented; fitted only in `bench/` , never in the API
Decision thresholds	`IMPL`	`decide.py:106-129`	0.85 / 0.50 + contradiction short-circuit
SHA-256 audit chain	`IMPL`	`evidence/audit.py`	Tamper-**evident**; verified over 900+ records
Evidence retraction	`IMPL`	`evidence/integrity.py`	Replaced hard delete; rescoring included
Provenance chain	`IMPL`	models + `warc_writer.py`	source → observation → artifact → evidence
Identifier extraction	`IMPL`	`extraction/extractor.py`	7 regex patterns; no checksum validation
XMR hard abstention	`IMPL`	`extractor.py:35`	Detects then refuses to score
Stylometry (classical)	`IMPL`	`stylometry/verify.py`	Cosine + Burrows's Delta
Short-text abstention	`IMPL`	`episodes.py:12`	< 50 words → abstain; 100% rate in benchmark
SimHash clone detection	`IMPL`	`clone_detector.py`	64-bit, MD5 token hashing
Favicon MurmurHash3 pivot	`IMPL`	`onion_probe.py:16`	Shodan-compatible hash
UTXO co-spend clustering	`IMPL`	`attribution/financial.py`	Union-find over input sets
Copilot (deterministic)	`IMPL`	`packages/copilot/`	16 ledger tools; refuses unknown entities
Copilot (Claude path)	`UNTESTED`	`copilot/llm.py`	Written but never executed — no SDK, no API key present
STIX 2.1 / PDF / CSV / JSON export	`IMPL`	`stix_export.py` , `reporting.py`	All four verified working

Feature	Status	File / module	Notes
Graph visualisation	IMPL	`GraphExplorer.tsx`	21 nodes / 31 edges on seed data
Neo4j projection	OPTIONAL	`graph/projection.py`	Degrades gracefully; not running by default
Node2Vec embeddings	EXP	`graph_embedding.py`	Trains per request, 3 epochs, hardcoded adjacency, unreachable from UI
Neural stylometry	UNTRAINED	`stylometry/neural.py`	**Random weights, never trained.** A fixed random projection
Candidate generation	NOT WIRED	`candidate_gen.py`	Implemented and tested; no caller outside tests
pgvector similarity	NOT IMPL	—	Docstring mention only; no vector column
MinHash	NOT IMPL	—	On the slide; absent from the repo (SimHash is what exists)
WARC (ISO 28500)	PARTIAL	`warc_writer.py`	Computes SHA-256 over artifacts; **does not write WARC files**
MinIO object storage	PLANNED	`docker-compose.yml`	No project code imports it
GitHub Actions CI	NOT IMPL	—	On the slide; no `.github/workflows/`
RBAC enforcement	NOT IMPL	—	`role` stored and issued but never checked
MFA	NOT IMPL	—	`mfa_enabled` stored but never verified

8.2 Source-of-Truth File Map

Concept	File	Function / symbol
Item LLR from priors	`packages/common/types.py`	`EvidenceItem.get_effective_llr()` — line 53
Family caps	`packages/common/types.py`	`FAMILY_CAPS` — line 23
m/u priors	`packages/attribution/mu_table.yaml`	`features:` , `contradictions:`
Fusion + discounting + caps	`packages/attribution/fusion.py`	`LLRFusionEngine.fuse_evidence()`
LLR → probability	`packages/attribution/calibration.py`	`sigmoid_llr_to_prob()` , `IsotonicCalibrator`
Decision thresholds	`packages/attribution/decide.py`	`evaluate_attribution()` — line 106
Cosine / Burrows's Delta	`packages/stylometry/verify.py`	`compute_cosine_similarity()` , `compute_burrows_delta()`
Abstention threshold	`packages/stylometry/episodes.py`	`MIN_WORD_COUNT_THRESHOLD = 50`
Jaccard neighbour overlap	`packages/attribution/candidate_gen.py`	`graph_topological_similarity()` — line 149
SimHash	`workers/extraction/clone_detector.py`	`SimHashCloneDetector.compute_simhash()`
Regex extraction	`workers/extraction/extractor.py`	`ExtractionEngine` — lines 16–22
Favicon hash	`workers/collection/onion_probe.py`	`OnionProbeEngine.inspect_favicon()`
UTXO clustering	`packages/attribution/financial.py`	`build_utxo_clusters()`

Concept	File	Function / symbol
Audit chain	`packages/evidence/audit.py`	`compute_entry_hash()`, `verify_audit_chain()`
Password hashing / JWT	`packages/evidence/auth.py`	`hash_password()`, `create_access_token()`
Ledger integrity	`packages/evidence/integrity.py`	`check()`, `repair()`, `rescore_hypothesis()`
All 38 endpoints	`apps/api/main.py`	`@app.get` / `@app.post` decorators
19 tables	`apps/api/database/models.py`	SQLAlchemy declarative models
Benchmark	`bench/report.py`	`run_benchmark_eval()`

8.3 Formula Sheet

Formula	Meaning	Range	Where used
$P(A\|B) = P(A \cap B) / P(B)$	Conditional probability	[0,1]	Conceptual foundation
$P(H\|E) = P(E\|H)P(H) / P(E)$	Bayes' theorem	[0,1]	Justifies the LR form
LR = $P(E\|H_1) / P(E\|H_0)$	Likelihood ratio	(0,∞)	Fellegi–Sunter basis
LLR = $\ln(m/u)$	Log-likelihood ratio	(−∞,∞)	`get_effective_llr()`
$\text{LLR}_{\text{eff}} = \ln(m/u) \cdot r \cdot c$	Reliability-weighted LLR	(−∞,∞)	`types.py:67`
contrib = λ·LLR (non-max in group)	Dependence discount	λ=0.25	`fusion.py:158`
scale = $\text{cap}_f / \Sigma\text{contrib}_f$	Family cap rescaling	(0,1]	`fusion.py:168`
$\text{LLR}_{\text{final}} = \Sigma\min(\Sigma c_f, \text{cap}_f) - \Sigma W_c$	Complete fusion	(−∞,∞)	`fuse_evidence()`
$P = 1/(1+e^{-(\text{LLR}+\text{prior})})$	Sigmoid calibration	(0,1)	`calibration.py:13`
cos(θ) = **A**·**B**/(‖**A**‖‖**B**‖)	Cosine similarity	[−1,1]	`verify.py:13`
$\Delta = (1/n)\Sigma\|z_i^A - z_i^B\|$	Burrows's Delta	[0,∞)	`verify.py:24`
$J = \|N(A) \cap N(B)\| / \|N(A) \cup N(B)\|$	Jaccard overlap	[0,1]	`candidate_gen.py:160`
sim = 1 − (Hamming/64)	SimHash similarity	[0,1]	`clone_detector.py:45`
entry_hash = SHA256(seq‖prev‖…)	Audit chain link	256-bit	`audit.py:97`
$S = \max(S_{\text{pgp}}, S_{\text{handle}}, S_{\text{vec}}, S_{\text{fav}})$	Candidate ranking	[0,1]	`candidate_gen.py:245`

8.4 Glossary

Abstention

> Deliberately declining to score when evidence is insufficient. In NETRA-X: stylometry below 50 words, and all Monero addresses. Returns exactly 0.0, not a guess.

Attribution

> Determining responsibility for activity. NETRA-X performs persona-level attribution (linking identities), not identification of a named human.

Base58

Bitcoin's encoding alphabet, excluding `0`, `O`, `I` and `l` to prevent visual confusion.

Brier score

Mean squared error of probabilistic predictions. Lower is better. NETRA-X: 0.1007 on synthetic data, reported but not gated.

Burrows's Delta

Authorship distance: mean absolute difference of z-scored function-word frequencies.

Calibration

Making stated confidence match observed accuracy. When a calibrated system says 80%, it is right about 80% of the time.

Candidate generation

Cheaply narrowing which pairs are worth expensive comparison. Avoids $O(n^2)$. Implemented but not wired.

Contradiction

Evidence that actively disproves a link. Uncapped and short-circuits the decision.

Dependence group

A label marking evidence items that reflect the same underlying leak. Within a group, only the strongest counts fully.

ECE

Expected Calibration Error — average gap between confidence and accuracy. NETRA-X: 0.0044 on synthetic data.

Evidence family

One of seven categories, each with a cap: EXACT_IDENTITY, FINANCIAL, INFRASTRUCTURE, CONTENT_NLP, STYLOMETRY, TEMPORAL, SEMANTIC_HANDLE.

Fellegi–Sunter

The 1969 probabilistic record-linkage framework using m/u parameters. The theoretical basis of the engine.

Hidden service

A server reachable only through Tor whose location is concealed. Addressed by a 56-character v3 onion address derived from its public key.

LLR

Log-likelihood ratio, $\ln(m/u)$. Positive supports the link, negative opposes, zero is uninformative.

m / u

$m = P(\text{signal} \mid \text{same operator})$; $u = P(\text{signal} \mid \text{different operators})$. In NETRA-X these are elicited, not learned.

MurmurHash3

Fast non-cryptographic hash. Used for favicons because Shodan indexes it, enabling clearnet pivots.

Posterior

Probability after weighing evidence — the system's output.

Prior

Belief before evidence. Here `prior_odds_log = -2.0`, roughly 12%.

Provenance

The recorded history of evidence: origin, time, lawful basis, and integrity proof.

Retraction

Withdrawing evidence while keeping the record. Replaced hard deletion.

ROC-AUC

> Probability a random positive outranks a random negative. NETRA-X: 0.9028 on synthetic data.

SimHash

> Locality-sensitive hash where similar documents yield similar fingerprints. **Not MinHash.**

STIX 2.1

> Structured Threat Information eXpression — the standard interchange format for threat intelligence.

Tamper-evident

> Modification is detectable. Distinct from tamper-*proof*, which prevents modification. NETRA-X is the former.

UTXO

> Unspent Transaction Output. Co-spending several UTXOs in one transaction implies common key control.

Waterfall

> The per-item breakdown showing how each evidence item contributed to the final score.

8.5 Viva Question Bank

Fundamentals

Q What problem does NETRA-X solve?

A It links anonymous dark-web personas that belong to the same operator, using evidence those operators leaked, and attaches a calibrated confidence and a full evidence trail to every link — so an analyst can judge the claim rather than trust it.

Q Does NETRA-X break Tor?

A No, and deliberately so. It performs no traffic correlation, runs no relays, and exploits nothing. It works entirely on published or accidentally exposed information — reused keys, reused wallets, copied favicons. This keeps it lawful and its output defensible.

Q Why is a rule engine insufficient?

A Four reasons: it treats all evidence as equal, cannot accumulate weak signals, double-counts correlated evidence, and cannot express uncertainty. A shared PGP fingerprint and a shared timezone both produce "link", though one is near-proof and the other is noise.

Q What is a v3 onion address?

A Exactly 56 base32 characters (`[a-z2-7]`) plus `.onion` , encoding an Ed25519 public key, checksum and version byte. The address *is* the key, so it cannot be seized from a registrar.

Mathematics

Q Derive the LLR and explain each term.

A LLR = $\ln(m/u)$, where $m = P(\text{evidence} \mid \text{same operator})$ and $u = P(\text{evidence} \mid \text{different operators})$. The ratio form comes from comparing two hypotheses over the same evidence, which cancels $P(E)$ from Bayes' theorem. The logarithm converts multiplication of independent ratios into addition, keeps values numerically stable, and makes supporting evidence positive and opposing evidence negative.

Q Why logarithms specifically?

A Three reasons: independent likelihood ratios multiply, and logs make that a sum that can be capped, discounted and explained per item; the PGP ratio alone is ~10^8 and chaining such values overflows; and logs give a natural zero for uninformative evidence with sign indicating direction.

Q What is dependence discounting and why is it necessary?

A Adding LLRs is valid only for independent evidence. A reused wallet address and a co-spending cluster containing it are one leak observed twice. Within a dependence group the strongest item counts fully and each additional item is multiplied by $\lambda = 0.25$. Without it, one leak observed five ways would look five times as identifying as it is.

Q Why is the prior −2.0?

A It corresponds to prior odds of $e^{-2} \approx 0.135$, about a 12% prior probability that two arbitrary personas are the same operator. It shifts the sigmoid so zero evidence yields $P \approx 0.12$ rather than 0.5 — encoding that most persona pairs are unrelated.

Q What LLR is needed for a high-confidence link?

A Posterior ≥ 0.85 with prior −2.0 requires LLR ≈ 3.73. Note the STYLOMETRY cap is 3.0 and TEMPORAL is 2.0 — so neither can reach the threshold alone. Corroboration across families is structurally required.

Machine learning — the hard questions

! What machine-learning model does NETRA-X use?

A **None in the live inference path.** The engine is a Bayesian likelihood-ratio calculator with expert-elicited priors. There is no `load_state_dict`, no `torch.load`, and no model artifact in the repository. An `IsotonicCalibrator` exists and can be fitted, but `.fit()` is called only in `bench/report.py`, so the API always falls back to the sigmoid.

! You have a neural stylometry module. What was it trained on?

A Nothing. `NeuralStylometryEncoder` is instantiated with `torch.manual_seed(42)` and no weights are ever loaded. Mathematically it is a fixed random projection of character-n-gram indices into 128 dimensions — closer to locality-sensitive hashing than to trained stylometry. It is deterministic and reproducible, but it has learned nothing about authorship.

Q Why not train a model?

A Supervised learning needs labelled pairs with ground truth "same operator". That data is held by law enforcement, is classified, and arrives only after arrests. Fitting to synthetic labels would learn the generator's assumptions rather than reality. Fellegi–Sunter exists precisely for principled linkage when parameters must be elicited rather than learned.

Q Where do the m and u values come from?

A Hand-authored in `packages/attribution/mu_table.yaml` — 11 features and 2 contradictions. They are expert judgements informed by the nature of each identifier (a 160-bit hash essentially never collides; timezones collide constantly). They are unvalidated against real-world frequencies, and the design isolates them in YAML precisely so they can be replaced with fitted values without touching the engine.

! Your ROC-AUC is 0.9028. Is the system 90% accurate?

A No, on two counts. ROC-AUC is not accuracy — it is the probability that a random true pair scores above a random false pair. More importantly, it was measured on 30 synthetic cases generated by `bench/synthetic/scenarios.py` with `seed=42`. The system is being evaluated on data produced by the same assumptions that built the scorer, so this measures internal consistency, not field accuracy.

! The benchmark prints "all metrics passed". Did they?

A The gated metrics passed: ECE 0.0044, FAR 0.00%, Recall@10 100%, ROC-AUC 0.9028. But Brier (0.1007) is printed and never asserted — the exit gate at `bench/report.py:94` checks only the other four. A commonly cited target of < 0.08 is not met, and the harness still reports success.

Trick questions

! Is a confidence score of 0.92 a 92% probability the attribution is correct?

A **No.** It is the output of a sigmoid applied to a summed LLR with a fixed prior of −2.0. It would be a true probability only if the m/u priors were empirically correct, the independence assumptions held, and the calibrator were fitted on representative labelled data. None of those conditions is currently met. It is best described as a *calibrated-form confidence score* — monotonic and comparable between hypotheses, but not a validated frequency.

! Your audit log is tamper-proof, correct?

A **Tamper-evident, not tamper-proof.** Anyone with database write access can delete rows or rebuild the whole chain. What they cannot do is alter one historical row and have verification still pass. True tamper-proofing needs external anchoring — publishing periodic root hashes to a third-party append-only log.

! Is the evidence ledger immutable?

A It is append-only by policy and now enforced. Originally `is_immutable` was decorative — nothing read it — while a hard DELETE endpoint removed rows, leaving 36 of 69 citations dangling across 13 of 14 hypotheses. Deletion is now retraction: the row stays, is marked withdrawn with reason and actor, stops counting toward scores, and affected hypotheses are rescored.

! The slide says MinHash. Where is it?

A It is not in the repository. The implementation is **SimHash** in `workers/extraction/clone_detector.py`. They are different algorithms: MinHash estimates Jaccard similarity over sets; SimHash estimates cosine similarity over weighted features via Hamming distance. The slide is wrong and should be corrected.

! Your candidate scoring uses weighted signal fusion — what are the weights?

A There are none. `multi_modal_candidate_search()` takes the **maximum** across signals, not a weighted sum. That is defensible for a recall stage — one strong signal is sufficient reason to consider a pair, and max prevents dilution by absent signals — but it is not the weighted formula design documents describe. The docstring also claims graph topology is included; the implementation has only four steps and never calls it.

! Which database does the system run on?

A SQLite by default — `DEFAULT_DB_URL_SYNC = "sqlite:///./netrax.db"`. PostgreSQL 16 is defined in `docker-compose.yml` and selected by setting `DATABASE_URL`, but no default path uses it. The architecture slide showing PostgreSQL overstates the default.

! Can this evidence be used in court?

A Not as it stands, and the system should not claim so. Admissibility depends on jurisdiction, collection legality, chain of custody and expert testimony — not on a hash chain existing. What the system provides is a defensible, reproducible, fully traceable analytical product suitable as an investigative lead.

Architecture and engineering

Q Why are contradictions uncapped when support is capped?

A Deliberate ethical asymmetry. A false positive is an accusation against the wrong party; a false negative merely continues an investigation. A −20 penalty can therefore overwhelm any accumulated support, so the system fails toward silence.

Q Why does the review endpoint map ACCEPT to ACCEPTED?

A A decision is a verb the analyst performs; a status is the state it leaves behind. Assigning the verb directly to `Hypothesis.status` put rows in state "ACCEPT", which no filter tab and no badge colour matched — an accepted hypothesis vanished from its own tab. The review row keeps the verb; only the state is mapped.

Q What happens to a score when evidence is retracted?

A `rescore_hypothesis()` recomputes the LLR by summing only surviving contributions and re-derives the posterior. Stored per-item contributions are reused rather than re-running fusion, because they are already the engine's post-discount, post-cap output. A hypothesis an analyst already ruled on does not silently revert to the queue.

Q Why is torch an optional dependency?

A It is very heavy and only two experimental modules need it. Importing it at package `__init__` level breaks every environment without it — including deployment — even for code paths that never touch it. `packages/attribution/__init__.py` and `packages/stylometry/__init__.py` guard the imports and substitute stubs that raise clearly if called.

Q How does the copilot avoid hallucinating?

A The deterministic path composes answers only from ledger query results and refuses when nothing matches. The optional Claude path receives *tools and no facts*, so every claim must come back from a real database call, and the system prompt requires abstention. Note the Claude path is written but untested — no SDK or key is present in this environment.

Limitations

Q How would an informed adversary defeat this?

A Easily, and this is the honest headline limitation. Generate a fresh PGP key per persona; never reuse a wallet address; vary writing style or use a translator; serve distinct favicons and certificates per host; post outside a consistent schedule. The model assumes non-adaptive adversaries who make operational mistakes. It also cannot detect deliberately planted false evidence — copying a rival's favicon to manufacture a link.

Q What is the biggest weakness of the mathematics?

A The independence assumption across families. Dependence discounting handles correlation *within* a group, but nothing discounts *across* families — and infrastructure and content signals are plausibly correlated, since a cloned site shares both. Scores may therefore be systematically inflated for cloned-infrastructure cases.

Q What would you fix first with more time?

A Wire the isotonic calibrator into the live path and fit it on real labels, since the product currently ships an uncalibrated sigmoid while the benchmark measures a calibrated one. Second, add checksum validation to the extractors. Third, enforce RBAC — the role is stored and issued but never checked.

8.6 Rapid Revision

NETRA-X in five minutes

NETRA-X links dark-web personas belonging to the same operator. It does not attack Tor; it reasons about mistakes operators already made — reused PGP keys, reused wallets, copied favicons, consistent writing style. Each shared signal becomes a log-likelihood ratio, $\ln(m/u)$, read from a hand-authored prior table. Correlated evidence within a dependence group is discounted by $\lambda = 0.25$. Each evidence family is capped, so no single kind of evidence can carry an attribution alone. Contradictions are uncapped and reject outright. The summed

LLR passes through a sigmoid with prior −2.0 to a posterior; ≥ 0.85 is high confidence, ≥ 0.50 low, below that insufficient. A human analyst makes every final decision, and every action is written to a SHA-256 hash-chained audit log.

Fifty things to remember

Fact	Fact
1. LLR = ln(m/u)	26. 19 database tables
2. m = P(signal | same operator)	27. 38 REST endpoints
3. u = P(signal | different)	28. 115 tests, ~12s runtime
4. λ = 0.25 dependence discount	29. 12,073 LOC Python
5. Strongest in group counts fully	30. 4,868 LOC TypeScript
6. EXACT_IDENTITY cap 10.0	31. Fellegi–Sunter, 1969
7. FINANCIAL cap 7.5	32. SQLite is the default DB
8. INFRASTRUCTURE cap 5.0	33. PostgreSQL is opt-in
9. CONTENT_NLP cap 5.0	34. Neo4j optional, degrades
10. STYLOMETRY cap 3.0	35. No trained model in the live path
11. TEMPORAL cap 2.0	36. Neural encoder is untrained
12. SEMANTIC_HANDLE cap 2.0	37. Isotonic fitted only in bench
13. Contradictions are uncapped	38. Candidate gen not wired
14. pgp_key_conflict = −20	39. SimHash, not MinHash
15. temporal_impossible = −15	40. No pgvector column exists
16. Prior log-odds = −2.0	41. No GitHub Actions workflow
17. P ≥ 0.85 → HIGH	42. RBAC stored but not enforced
18. P ≥ 0.50 → LOW	43. Tamper-evident, not tamper-proof
19. Contradiction short-circuits	44. Retraction replaced deletion
20. PGP fingerprint LLR = 18.4	45. Brier 0.1007, not gated
21. Timezone LLR = 1.9	46. ROC-AUC 0.9028 on synthetic data
22. Stylometry abstains < 50 words	47. ECE 0.0044, FAR 0.00%
23. XMR always abstains	48. 100% short-text abstention
24. Onion address = exactly 56 chars	49. All seed actors are synthetic
25. Favicon hash = MurmurHash3	50. Human decides, machine proposes

THE SINGLE MOST IMPORTANT THING TO SAY IN A VIVA

NETRA-X is not a system that decides who someone is. It is a system that makes the evidence for a claim **legible, weighted and reviewable** — and that refuses to answer when the evidence does not support one. Every number it displays traces to a row, every row traces to a hashed artifact, and every conclusion requires a human to sign it. Where the system is weak, this document says so, because a project that knows its own limitations is more credible than one that claims none.